

Reflections on the design, applications and implementations of the normative specification language eFLINT

L. Thomas van Binsbergen^a, Christopher A. Esterhuyse^a, Tim Müller^a

^a*Informatics Institute, University of Amsterdam, Science Park 900, 1098 XH, Amsterdam, The Netherlands*

Abstract

Checking the compliance of software against laws, regulations and contracts is increasingly important and costly as the embedding of software into societal practices is becoming more pervasive. Moreover, the digitalised services provided by governmental organisations and companies are governed by an increasing amount of laws and regulations, requiring highly adaptable compliance practices. A potential solution is to automate compliance using software. However, automating compliance is difficult for various reasons. Legal practices involve subjective processes such as interpretation and qualification. New laws and regulations come into effect regularly and laws and regulations, as well as their interpretations, are subjected to constant revision. In addition, computational reasoning with laws requires a cross-disciplinary process involving both legal and software expertise.

This paper reflects on the domain-specific language eFLINT developed to experiment with novel solutions to these challenges. Specifically, the language has been developed to experiment with the abstract syntax and semantics of a language supporting different types of reasoning for various applications. The language combines declarative and procedural elements, formalises connections between legal concepts and computational concepts, and is designed to automate compliance checks before, during and after a software system runs. The various design goals and applications areas for the language give rise to (conflicting) requirements. This paper presents and reflects on the current design of the language by recalling applications and requirements. As such, this paper reports on results and insights of an investigation that can benefit language developers within the field of automated compliance.

This is the arXiv version of the paper published by Elsevier under a modified title at <https://doi.org/10.1016/j.cola.2026.101411>.

1. Introduction

Laws, regulations, contracts, and other *social policies* serve to regulate social systems by establishing *norms* that determine how actors within the system are expected to behave. A wide range of automated techniques is available to aid in the enforcement of such social policies within software systems. In distributed software systems, *system policies* are widespread as means to regulate the behaviour of system components, separating the description of the policy from the implementation of the components to enhance adaptability and transparency. For example, the XACML access control language is designed to regulate access to resources through access control policies that can easily be reconfigured when relevant social policies change [88]. As another example, contracts encode how parties' rights and obligations evolve over time in response to events and can naturally be formalised as state machines. Smart contracts implement such state machines to manage the exchange of digital assets [108], akin to the automation

of the execution of a contract [54] without the need of an intermediary. However, both examples do not directly embed normative concepts such as obligations and power, limiting their expressivity with respect to social policies in general. Moreover, there is still a significant conceptual gap between the social policy text and its encoding in a smart contract or access control policy. Existing languages that close this gap by embedding normative concepts are designed for specific subsets of the law. For example, Symboleo [105] is used to formalise and execute contracts whereas Catala [83] targets computational laws such as tax laws.

A general solution to automating compliance that is tenable from the legal perspective needs to address several requirements. Firstly, a formal representation of policies is required that is sufficiently general to capture policies from a variety of sources and at different levels of abstraction. For example, the GDPR privacy regulation is more abstract than an organisational policy or a data sharing agreement.

Secondly, the solution should enable different kinds of reasoning. For example, conformance checking can be used to determine whether an implemented process is compliant with policies based on event logs [2]. Model checking [30] can be used to gain confidence in the correctness of a policy specification. Simulation helps determine the effects enforcing specific policies on systems or their constituent

*Corresponding author

Email addresses: ltvanbinsbergen@acm.org (L. Thomas van Binsbergen), c.a.esterhuyse@uva.nl (Christopher A. Esterhuyse), t.muller@uva.nl (Tim Müller)

parts, including human stakeholders. Search can be used for policy-aware planning and optimisation.

Thirdly, the solution should embrace the fact that legal processes, such as interpretation and qualification, are inherently subjective, and that compliance depends on (in)actions by users. These observations render it impossible, from a legal perspective, that a system can prevent all violations by design without adaptation or response mechanisms. This is due to the fact that: (a) the same legal norm may be interpreted differently by different stakeholders, (b) conflicting legal norms may simultaneously apply to a case, (c) norms may be interpreted differently and situations may be qualified differently depending on the specifics of a case, and (d) legal obligations may require actions by users that cannot be technically enforced or are not automatically enforced to preserve the users freedom of choice. Therefore, solutions are needed in which violations within the system can occur and can be observed, together with penalty- or compensation-mechanisms to respond to non-compliance. An extensive overview of the challenges related to compliance is given by Hashmi et al. [58].

This paper reports on intermediate findings of an investigation into a general solution to automating compliance. Central to the investigation is the design and implementation of eFLINT, a domain-specific language (DSL) for developing executable specifications of norms [15]. The language supports various types of reasoning and can be applied at different stages in the software development process. The design is based on powerful software engineering concepts such as modularity and extensibility and combines concepts from the declarative, logical, and imperative programming paradigms. The language has been applied in various experiments demonstrating: assessing individual cases [15], bounded model checking [42], run-time compliance checking [81], normative multi-agent systems [91], and generating access control policies from social policies [14]. Over time, the language has evolved to become more flexible and suitable for use within these various application areas. However, the language is oriented towards software experts and is not suitable for use by legal experts. Instead, the language is intended as an intermediate language that can provide semantics to a surface language such as FLINT [37, 22], for which it was first developed.

In this paper, we reflect on the design of the language using requirements extracted from applications of the language. The reflection is based on 11 software systems that incorporate eFLINT to reason about compliance. The systems are developed by students, researchers and practitioners and demonstrate the effectiveness of the language at various technological readiness levels. Each system provides either validation of the language in a lab environment or in a realistic test environment with real stakeholder interactions. In one case, the system provides an operational service maintained by an industrial partner. These validation experiments are reported in 8 peer reviewed papers, two theses and one white paper (see Section 7). Through these reflections, this paper is intended to be insightful for

researchers interested in legal DSLs, norm specification languages, normative reasoning, and automating compliance. Specifically, this paper contributes:

- A list of requirements identified through a range of applications of the language such as static and dynamic compliance checking, verification and planning.
- A definition of the latest version of the eFLINT language together with a reflection on the design decisions in relation to the identified requirements. This paper is the first to define the language’s extension with *physical actions* and *open types*.
- A runtime performance comparison between two implementations of the language: a bespoke reference interpreter and an interpreter that employs Clingo’s answer-set solving.

The development of the language went through various phases during which additional applications were targetted and requirements were identified. Rather than attempting to present these as isolated, well-defined phases, we instead present the current design followed by a reflection on the language’s evolution, application areas and the requirements identified along the way.

The paper is organised as follows. Section 2 motivates and gives a high-level overview of the background concepts and terminology most influential on the design. Section 3 introduces the language through a running example. Section 4 defines the abstract syntax and discusses the semantics of the language as it exists today alongside a brief overview of the evolution of the language. Section 5 analyses specific design decisions and trade-offs that have been made to address requirements identified throughout the language’s evolution. Section 6 motivates and compares the two most prevalent implementations and includes a runtime performance analysis. Section 7 discusses specific experiments with the language in various application areas that give evidence to the suitability and wide applicability of the language. In doing so, Section 7 reflects on the identified requirements, discusses the main limitations of the current design, and suggests future directions. Related work is discussed in Section 8 and Section 9 concludes.

2. Background

This section introduces the main concepts and terminology underpinning the design of eFLINT at a high-level of abstraction. The section covers process modelling, logic programming and norm representation.

2.1. Process Modelling

Process modelling languages show variety in the purposes for which they are developed and their visual or notational style. For example, Business Process Model and Notation (BPMN) is used to model business process using flow-charts [1]. In the context of software engineering, the

activity diagrams of the Unified Modeling Language (UML) are used to describe the behaviour of a system or system component [80]. On the other hand, languages oriented towards formal analysis, such as nuXmv [25] and Alloy [69], tend to use a textual notation. Process modelling languages have in common that their semantics can be expressed using event-based mathematical models involving states and transitions. Examples are petri nets, automata and transition systems. The choice of semantic model may depend on whether the modelled system has inherent concurrency or parallelism. Another factor is whether the system can reach infinitely many states or process infinitely many events.

The semantics of eFLINT is described in terms of transition systems in [15] and Section 4. This approach gives the flexibility to a model a broad range of systems, including systems with infinite, concurrent and bounded parallelism. For bounded model checking, assumptions can be placed on norm specifications to ensure the underlying transition system has the required, finite structure [31]. Relevant extensions to the semantics of eFLINT that have not yet been considered include additional types of primitives beyond Booleans and integers, probabilistic reasoning and continuous elements, e.g., to represent clocks.

2.2. Regulatory Compliance

In the context of business process modelling, Groefsema et al. define ‘verification relations’ for checking the adherence of system designs and implementations with respect to different types of requirements [57]. In that work, *compliance* is defined as a relation between a requirement specification and the design of a system, captured by a process model. On the other hand, *conformance* is a relation between the events logs produced by an enacted process and a requirement specification or design model. These relations are used to identify ‘verification techniques’ by also considering the moment in the software development lifecycle the checks are performed. The verification techniques that check adherence with regulatory requirements are *regulatory compliance checking* and *auditing*.

Auditing is verifying the conformance relation between event logs and a formal specification of the regulatory requirements. Auditing is an after-the-fact (hereafter: *ex-post*) technique that is applied to complete event logs, typically from a certain time period. Process mining enables the construction of a process model from the logs [111] as a first step towards verifying the conformance relation.

Regulatory compliance checking has a static and dynamic counterpart. The static variant verifies the compliance relation between the model of a system design and the regulatory requirements. Such a before-the-fact (hereafter: *ex-ante*) technique serves to prevent non-compliance as much as possible. However, dynamic and ex-post techniques are needed in practice as checking the adherence with certain regulatory requirements might require runtime information, e.g., about user actions and inactions. Moreover, the regulatory requirements might themselves be dynamic in nature. Dynamic regulatory compliance

checking verifies the conformance relation between event logs and regulatory requirements at runtime.

Section 7 gives an overview of the experiments we performed to demonstrate the use of eFLINT for these types of verification. In addition, we have demonstrated *adaptability* of systems through the integration of eFLINT.

2.3. Runtime Integration

In runtime verification, a component or subsystem is placed under monitoring [12]. The observed event traces are compared to a behaviour specification typically comprised of a process model describing the intended/permitted behaviour. Such a behaviour specification can also encode regulatory requirements [26]. Observed violations can be used as input to a self-adapting system’s MAPE-K loop [74, 20] or as part of norm revision [35].

In distributed data processing systems, orchestrators assign tasks to worker nodes in fulfilment of a data processing workflow [110]. Orchestrators plan the workflow by identifying and distributing tasks in adherence to given constraints, which may include regulatory requirements.

In access control, data requests are intercepted to ensure data is accessible only to actors with explicit authorization. The XACML access control architecture [88] defines technical roles for administering, enforcing and deciding on access control policies. The primary benefit of the separation is that it gives system administrators the ability to update policies without modifying the system’s implementation.

In our experiments, eFLINT is integrated in a system by the implementation of an API for interacting with an underlying eFLINT interpreter. In what follows, we refer to a component implementing such an API as a *reasoner*.

2.4. Logic Programming

Process modelling is used by eFLINT to capture the temporal evolution of the state of a system. To represent information within the state, eFLINT relies on concepts familiar from logic programming. Specifically, the user defines sets and relations with predicates determining for every possible element whether it is present in the set/relation according to the current state. In this context, the state is also referred to as a *knowledge base*. This style of knowledge representation is common in logic programming languages such as Prolog [32], Clingo [48] and Datalog [27], and in ontology languages such as OWL [63, 78] and RDF [75]. From this paradigm, eFLINT has also adopted *Horn-clauses*; if-then rules for deriving new predicate *instances* from existing predicate instances. For example, the ‘counts as’ relation is commonly used for sub-typing in ontological reasoning and is easy to implement using eFLINT’s derivation rules. For example, “if an object *X* is a car, then *X* is also a vehicle”:

```
Fact car           // a predicate recognising cars
Fact vehicle       // a predicate recognising vehicles
Holds when car(vehicle) // cars ‘count as’ vehicles
```

The semantics of logic programming languages differ in how they treat Horn-clauses. For example, Datalog places

a restriction on the usage of variables that makes it possible to compute predicates efficiently. Prolog and Clingo are more expressive and require more powerful *solvers* such as Potassco for Clingo [50]. Especially relevant to this paper is Clingo and its stable model semantics [51] that enables the specific form of logical programming referred to as *answer-set programming* (ASP) [38]. A solver for ASP searches for all possible answers – sets of predicate instances that can consistently hold simultaneously according to the clauses.

Negation is a well-studied challenge in logic programming [29]. The use of *negation as failure* within eFLINT is discussed in detail in Section 5.3.

The event calculus [76, 99, 98, 28] has been studied extensively as a way to admit temporal reasoning in logics. Its discrete timepoints and explicitly evolving truth-assignment to predicate instances correspond naturally to a transition system semantics in which timepoints correspond to states. As predicate instances can evolve from being true to false, the event calculus naturally gives rise to a kind of defeasibility of facts, explored separately within defeasible logics [87, 55]. In [42], the semantics of eFLINT is formalised in terms of the event calculus on top of ASP.

2.5. Norm representation

An important distinction in normative languages is the ability to separate between what can physically be achieved and what is permitted by the norms, i.e., to separate *ability* and *permissibility*. An additional separation is needed between permissibility and the concept of institutional *power* as formalised by Jones and Sergot [70]. In essence, an actor can manifest a power by performing an action that has normative implications, e.g., signing a purchase agreement creates an obligation to pay. The absence of a permission does not necessarily imply the absence of a power [36]. An explicit goal for the design of eFLINT has been to support both the concept of institutional power and the deontic concepts of permission, prohibition and obligation. In general terms, these concepts correspond to those in the legal framework by Hohfeld [61, 60]. Note that in this framework the term *duty* is used to refer to the specific ‘obligation to act’ rather than the more general ‘obligation’. In this framework, all duties are associated with a ‘duty-claim’ relationship between a duty holder and a duty claimant who can claim benefits from the fulfilment of the duty. Similarly, all powers are associated with a ‘power-liability’ relationship between a performer and a recipient who receives the effects of the power when applied.

2.6. Calculemus-FLINT

The design of eFLINT was motivated to assess scenarios for compliance with norms specified in the Formal Language for the Interpretation of Normative Texts (FLINT) of Van Doesburg [36, 22]. The intention is for FLINT to be used as a surface language by legal experts and eFLINT as a back-end language by software experts.

The Calculemus-FLINT framework [37] provides a methodology for interpreting sources of norms to produce a

<i>Act frame</i>	«assisting with the contracting of a valid marriage»
<i>Actor</i>	[ordinary or priest ...]
<i>Object</i>	[marriage attempt]
<i>Interested Party</i>	[spouses]
...	...
<i>Creating postcondition</i>	[valid marriage]; ...
<i>Terminates postcondition</i>	[marriage attempt]

```

Fact [ordinary or priest ...]
Fact [spouses]
Bool [marriage attempt]
Bool [valid marriage]
Act <<assisting with the contracting
  of a valid marriage>>
  Actor      [ordinary or priest ...]
  Recipient  [spouses]
  Related to [marriage attempt]
  Creates    [valid marriage]()
  Terminates [marriage attempt]()

```

Figure 1: A FLINT and eFLINT specification of the same action-type (Act). This simplified example is taken from [36].

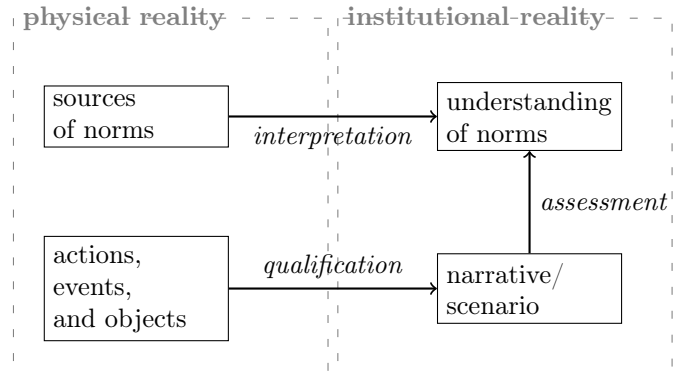


Figure 2: Schematic overview of the processes of interpretation, qualification and assessment as taken from [15] and based on Searle [103].

structured, formal interpretation. The method suggests to recognise actions, duties, actors and facts based on sentence structure and to fill action-, duty-, and fact-frames with text fragments extracted directly from the sentences. An example frame is shown at the top of Figure 1.

When complete, the resulting interpretation is used to assess a particular case for compliance against the formalised interpretation. However, the facts of the case also need to be established, through a process known in the legal domain as *qualification* [103]. The relation between the aforementioned processes is given in Figure 2. Note that interpretation and qualification are subjective, require legal expertise and may be the subject of disputes, e.g., in a courtroom. However, by formalising both the interpretation result and the details of the case, unambiguous assessment can be realised. By making FLINT executable – eFLINT abbreviates “executable FLINT” – assessment has been automated. An eFLINT specification has type declarations to give meaning to frames (see Figure 1) and includes a sub-language for defining scenarios.

Towards an intermediate language. A long-term goal for the development of eFLINT has been to experiment with the abstract syntax and semantics of an intermediate language for reasoning with arbitrary normative sources. The high-level design decisions described in previous paragraphs are motivated by this goal. The hypothesis has been that the language can be used to formalise arbitrary normative sources by adopting the most fundamental normative concepts. Moreover, the language can support different types of normative reasoning for different types of compliance checking by adopting a transition system semantics and logic programming. The language is intended as a step towards an intermediate language that can support multiple source languages, such as FLINT, and can translate into multiple targets, such as Clingo, for various applications.

Sections 3 to 5 give a thorough description of eFLINT. Reflections on the design, implementations, applications, and requirements are given in Sections 5 to 7.

3. Running Example

This section introduces the eFLINT language through an example related to auctioning. The code is introduced incrementally with code fragments building on top of each other in the order they are given. The reference interpreter for the language is available online [13].

Type-declarations introduce sets and relations, each with a domain whose instances receive a truth-assignment in a runtime state indicating whether the instance is an element of the corresponding set or relation. The following type declarations introduce the initially empty sets **bidder**, **object**, **price**, and **display** and the relations **bid** and **min-price-of**. The set **display** is declared with **Var**, indicating at most one instance can hold true for this type, i.e. **display** acts as a variable to which an instance is assigned (optionally). Similarly, the relation **min-price-of** is ‘functional’, i.e. it maps **objects** to a unique price as a mathematical, partial function.

```
Fact bidder Identified by String // any string
Fact object Identified by String
Fact price Identified by Int // any int
Var display Identified by object
Function min-price-of Identified by object * price
Fact bid Identified by bidder * object * price * int
```

The following *statements* define the function by asserting certain instances.

```
+min-price-of(Watch, 100).
+min-price-of(Clock, 200).
+min-price-of(Painting, 400).
```

Derivation rules can be added to type declarations to infer truth assignments from knowledge about (other) facts. The following *type extensions* demonstrate this:

```
Extend Fact object Derived from min-price-of.object
Extend Fact price Derived from min-price-of.price
, bid.price
```

The **Extend** keyword adds clauses to an existing declaration. The last line above determines that every price included in a bid is an instance of **price**. The results of multiple

derivation rules accumulate through set union. The second line thus determines that the asserted minimum prices are also instances of **price**. A derivation rule can be written as a Boolean expression using **Holds when**. The Boolean expression is evaluated in a context in which the fields of a type are bound as variables, i.e., **bidder** and **price** below.

```
Var highest-bid Identified by bidder * price Holds when
(Exists bid: bid.bidder == bidder && bid.price == price
&& bid.object == display.object
&& (Forall bid': bid'.price <= price
When bid'.object == display.object))
```

This definition assumes that the fields **bidder** and **price** and the variable **bid** can be enumerated to evaluate the expression for each combination of instances of these types.

Action-types are fact-types with instances – referred to as actions – that can be *performed*. The so-called *postconditions* determine the effects of performing actions. The action-type below has two fields, the implicit field **actor** and **object**. An instance of the type holds true when its actor is recognised as an **auctioneer**. A performed action that does not hold true raises a violation, but still has its effects. The effect is the creation of the instance **display(object)**, as well as the implicit termination of any other elements of **display** owing to its status as a **Var**.

```
Var auctioneer // responsible for displaying objects
Act start-bidding Related to object
Holds when auctioneer(actor)
Creates display(object)
Extend Fact actor Derived from auctioneer
```

To create (terminate) an instance is to make it true (false). As such, performing an action results in the transition from one state to another. This also holds for *events* – essentially actions without a performing actor. Whereas actions are ‘performed’, events and actions are collectively said to be *triggered*. Derivation rules are declarative in that they do not cause state transitions. Instead, they can be understood to ‘close’ a state by deriving new instances from the instances created and maintained (i.e., true and not terminated) by the previous transition.

The actor of an action can also be named explicitly:

```
Act place-bid Actor bidder Related to object, price
Holds when bidder
Conditioned by display(object), price >
Max(Foreach bid: bid.price When bid.object == object)
Creates bid(int =
Count(Foreach bid: bid When bid.object == object))
```

This action-type declaration uses a form of constructor application with implicit fields to create an instance of **bid**. The explicit field **int** distinguishes bids from the same bidder. The implicit fields **bidder**, **object**, and **price** are copied directly from the fields of the triggered instance of **place-bid**, i.e., **bidder** = **bidder**, **price** = **price**, etc. The **Conditioned by** clause establishes preconditions for instances to be *enabled* and actions to be *permitted*. For an instance to be enabled, it must hold true by being created or derived, and all its preconditions must hold. A prohibition is the negation of a permission and performing a prohibited action results in a violation. In this example, bids are only permitted on displayed objects and must involve a price higher than any previous bid.

A *physical* action, as opposed to the previous *institutional* actions, holds by default and is permitted unless it *synchronises* with an institutional action that is prohibited. Unlike an institutional action, a physical action can thus be disabled and permitted. The intuition is that conditions on physical actions constrain ability, not permission. The physical action-type below intuitively captures the *qualification* of a bidder raising their hand at an auction as placing a higher bid on the item currently on display.

```
Physical raise-hand Syncs with place-bid(
  bidder = actor, object = object, price =
  min-price-of.price + increment *
  Count(Foreach bid: bid When bid.object == object))
When bidder(actor) && display(object) &&
  (min-price-of.object == object)

Var increment Identified by Int
Extend Fact price
  Derived from min-price-of.price + increment
    ,bid.price + increment
```

The consequence of synchronising an action A with an action B is that A inherits all the pre- and postconditions of B, effectively triggering B whenever A is triggered. The ‘synchronises with’ relation is directional: in the above, **raise-hand** is not triggered if **place-bid** were to be triggered. The relation is transitive and potentially cyclic¹. In the above, the type **increment** is used as a parameter through which it is determined how much higher the current bid is compared to the previous. The additional derivation rules for **price** are needed to enable those specific instances of **place-bid** that can be triggered by the next occurrences of **raise-hand**.

A duty-type declaration defines a fact-type with mandatory fields for a duty-holder and a duty-claimant to establish a duty-claim relation in the Hohfeldian legal framework. A duty-type declaration has zero or more additional fields (such as **price** below) and zero or more violation conditions. A duty raises a violation when it is enabled and when one or more of its violation conditions hold.

```
Bool undue-payment-delay
Duty payment-duty Holder bidder Claimant auctioneer
  Related to price
  Violated when undue-payment-delay()
```

In the example, the duty to pay is created when the bidding on a particular object is ended by the auctioneer. That is, an auctioneer has the *power* to create payment duties.

```
Act end-bidding Holds when auctioneer(actor)
  Creates payment-duty(bidder=highest-bid.bidder
    ,price=highest-bid.price)
  Terminates display
    ,bid When bid.object == display.object
```

The **Terminates** clause refers to the variables **display** and **bid**. As they are not fields of **end-bidding**, the variables are implicitly bound by an occurrence of **Foreach**. As a result, all instances of **display** are terminated and all instances of **bid** are terminated for which hold that the object referred to in the bid is on display. The condition

on the termination of **bid** is realised by the application of the infix operator **When**. The notion of power can thus be constrained with the use of **When** within post-conditions. The termination of the **display** and **bid** facts are considered to occur simultaneously. The **Terminates** expressions are evaluated in the same context; the object is still on display in the second **Terminates** clause. The same is true for **Creates** and **Terminates** clauses and, when in conflict, the creation of a fact is favoured over its termination.

The fragment below shows a sequence of statements forming a *script*. A script includes assertions of creation (+) and termination (-), action/event triggers (without prefix), and queries (?). At the end of the script, Bob wins the auction being the last to raise a hand. Bob receives the duty to pay 140 for the watch because all bids but the first raise the minimum price of a 100 with increments of 10, following the definition of **raise-hand**. The query at the end confirms the existence of the duty.

```
+bidder(Alice). +bidder(Bob). +bidder(Chloe).
+auctioneer(David).
+increment(20).
start-bidding(David, Watch). // action trigger
raise-hand(Alice). raise-hand(Bob). raise-hand(Alice).
raise-hand(Chloe). raise-hand(Bob).
end-bidding(David).
?payment-duty(Bob, David, 100+4*increment). // True
```

4. Language Definition

This section introduces the syntax and semantics of the latest version of the language as implemented by the reference interpreter. The abstract syntax is defined formally in Figures 3 to 5. A concrete syntax definition can be found in the source code of the reference interpreter. The file **Parse.hs** contains an EBNF-like grammar description using the grammar combinators of [17]. The sections Sections 4.1 to 4.3 describes the syntax of expressions, declarations and top-level constructs informally. In parallel, the static and dynamic semantics of the language are described at a high-level. A formal semantics is out of scope for this paper. A formal treatment of the semantics of a subset of the language is found in [42]. Section 4.3 gives a rough overview of the language’s evolution.

4.1. Expressions

Figure 3 defines the abstract syntax of eFLINT expressions. Expressions are formed by string, Boolean and integer literals, variable occurrences and the application of various operators. Expressions are further distinguished into *instance expressions* and *Boolean expressions* based on the type of value they evaluate to. Boolean expressions are those evaluating to a Boolean value and are formed by the comparison operators and operators such as *exists*, *forall*, and *violated*. Instance expressions are those evaluating to a string, integer or tuple ‘tagged’ with a type identifier of a user-defined type, i.e., an instance *v* is an element of the sort **instances**. As the components of a tuple are tagged values, tuples can be seen as records with the tags as field

¹Cycles are easy to detect and resolve. Semantically, the transitively closed set of synchronised actions is triggered synchronously.

$d \in$	type-ids	$=$	...
$x \in$	vars	$\supseteq$	type-ids
$i \in$	$\mathbb{Z}$	$=$	...
$b \in$	$\mathbb{B}$	$::=$	tt ff
$s \in$	strings	$=$	...
$e \in$	elems	$::=$	$s \mid i \mid \text{tuple}(v_1, \dots, v_n)$
$v \in$	instances	$::=$	(e, d)
$t \in$	exprs	$::=$	$s \mid i \mid b \mid x$ $\mid \text{construct}_t(d, t^*)$ $\mid \text{construct}_m(d, m^*)$ $\mid \text{project}(t, x)$ $\mid \text{foreach}(x, t) \mid \text{when}(t_1, t_2)$ $\mid \text{exists}(x, t) \mid \text{forall}(x, t)$ $\mid \text{holds}(t_1) \mid \text{enabled}(t_1)$ $\mid \text{violated}(t_1)$ $\mid \text{eq}(t_1, t_2) \mid \text{geq}(t_1, t_2)$ $\mid \text{count}(t)$ $\mid \dots$ (basic operators omitted)
$m \in$	modifiers	$::=$	$\text{assign}(x, t)$

Figure 3: The abstract syntax of eFLINT expressions.

names. Instance expressions are formed by operators such as *construct*, *project*, and *foreach*. Expressions that evaluate to a string or integer literal without a tag are only acceptable as sub-expressions.

Type checking ensures that the right kind of expressions are used within expressions and declarations. Automatic coercions can be applied to correct any type mismatches. For example, the *holds* operator is automatically applied to an instance expression in places where a Boolean expression is expected. Sub-expressions evaluating to a string or integer literal are tagged with the correct type identifier in places where an instance expression is expected. Such coercions can only be applied in places where the expected type identifier can be inferred from context, e.g., within constructor applications. Coercions may also remove the tag, e.g., in a sub-expression where an integer is expected.

Type identifiers play multiple roles: besides giving a name to a type, they are also used as constructor names, field names, parameters and variables more generally. In the concrete syntax, fields, parameters and variables are type identifiers decorated with an index or with zero, one or more prime symbols. For example, the variables **object**, **object2**, **object'** refer to values of type **object**. This is formalised by the requirement that there is a projection $\text{proj} : \mathbf{vars} \rightarrow \mathbf{type-ids}$ from variables to type identifiers.

Constructor application appears in two forms. In both cases, a type identifier d is applied as a constructor to create a tuple tagged with the type d . The form construct_t expects as many expressions as arguments as the type d has fields, given in the order matching the declaration of the type. The form construct_m can receive fewer arguments, including none at all, written in any order. In this form, arguments explicitly associate an expression with a field of the type

d (see **modifiers**). Any field x not mentioned within an argument of the constructor application is implicitly defined through the modifier $\text{assign}(x, x)$. The projection operator $\text{project}(t, x)$ is as usual: it expects an expression t that evaluates to a tuple tagged with a type that has the field x . The value of that field is the result of the expression.

The *foreach* operator is given a variable x and an instance expression t and generates n bindings for x , one for each² possible instance v of the type $\text{proj}(x)$. The expression t is evaluated n times, each time in a modified context binding x to one of the instances v . The result is a sequence of $m \leq n$ instances. Instance expressions that are not guaranteed to return a single instance are only acceptable in certain places such as postconditions (e.g., **Create**) and as the immediate sub-expression of an accumulator such as **Count**. The operator $\text{when}(t_1, t_2)$ is available to filter out results in such expressions. The operator expects an instance expression t_1 and a Boolean expression t_2 and returns the value of t_1 , but only when t_2 evaluates to true. As both sub-expressions are evaluated within the same context,³ *when* serves as a guard to ensure that instances are returned only when certain conditions are satisfied. The operator can only be used in places where an expression is accepted that may return multiple instances.

The *exists* and *forall* operators are evaluated similarly to *foreach* except that they operate on Booleans. The evaluation of the sub-expression returns zero, one, or more Boolean values. The operator returns true if and only if one of these values is true, in the case of *exists*, or if all do, in the case of *forall*.

The operators *holds*, *enabled*, and *violated* are predicates that determine whether the instance computed from their sub-expression satisfies a certain property. An instance v is considered to *hold* if v is assigned ‘true’ in the input to the program, is true in the current state, or is derived within the current state (in that order of priority). An instance v is considered *enabled* if it holds and if all the preconditions (**Conditioned by** clauses) evaluate to true in the current context. An instance v is considered *violated* if it is an enabled instance of a duty-type for which one or more of its violation conditions evaluate to true.

The reference interpreter supports various comparison operators such as *eq* and *geq* for equality and ‘greater or equals’. In addition to **Count**, accumulator operators such as **Max** and **Sum** are available for integer expressions. The immediate sub-expression of an accumulator must always be an occurrence of *foreach*. Section 5.3 provides a detailed discussion on the semantics of the negation operator.

The language is statically typed. The type of all variables and user-defined types are known at their declaration and binding sites. This is in part due to the projection *proj* described earlier that maps a variable name to a type. A

²When the type $\text{proj}(x)$ has an infinite domain, only the instances of the type that hold true in the current state are enumerated.

³The context of expression evaluation consists of the set of active bindings and a set of truth-assignments to instances.

$\delta \in$	domains	::=	strings <i>string-set</i> (s^*) $\mathbb{Z}$ <i>int-set</i> (i^*) <i>tuples</i> (x^*)
$dc \in$	decls	::=	<i>fdc</i> <i>adc</i> <i>edc</i> <i>ddc</i>
$r \in$	restrs	::=	var function
$fdc \in$	fact-decls	::=	<i>fdecl</i> ($d, \delta^?, r^?$)
$ddc \in$	duty-decls	::=	<i>ddecl</i> (d, x_1, x_2, x^*)
$adc \in$	act-decls	::=	<i>adecl</i> (d, b, x_1, x^*)
$edc \in$	event-decls	::=	<i>edecl</i> (d, x^*)
$ext \in$	extensions	::=	<i>fex</i> <i>dex</i> <i>aex</i> <i>eex</i>
$fex \in$	fact-exts	::=	<i>ext-fact</i> ($d, fcl^*, domcl^*$)
$dex \in$	duty-exts	::=	<i>ext-duty</i> (d, dcl^*)
$aex \in$	act-exts	::=	<i>ext-act</i> (d, ecl^*)
$eex \in$	event-exts	::=	<i>ext-event</i> (d, ecl^*)
$domcl \in$	dom-clss	::=	<i>int-set</i> (i^*) <i>string-set</i> (s^*)
$fcl \in$	fact-clss	::=	<i>cond-by</i> (t) <i>deriv-from</i> (t) <i>open</i> () <i>close</i> ()
$dcl \in$	duty-clss	::=	<i>fcl</i> <i>viol-when</i> (t)
$ecl \in$	event-clss	::=	<i>fcl</i> <i>c</i> <i>syncs-with</i> (t)
$c \in$	post-conds	::=	<i>create</i> (t) <i>terminate</i> (t) <i>obfuscate</i> (t)

Figure 4: Abstract syntax of eFLINT declarations.

standard, static type checking algorithm can thus be used to type-check expressions and specifications. The language is not strongly typed. A literal value or tuple (see **elems**) can be used as an instance of more than one type. Runtime values are therefore tagged with types (see **instances**).

4.2. Declarations

The core of an eFLINT specification is a collection of type declarations and type extensions. The syntax of declarations and extensions are defined in Figure 4. There are four kinds of types that can be declared: facts, duties, actions and events. Each kind of declaration gives a name d to the type and associates a domain δ with the type.

In the case of a fact-type, this domain can be a set of string literals, integer literals, or tuples. A domain made out of a set of literals can be infinite (**strings** or $\mathbb{Z}$) or fully enumerated by the user (*string-set* or *int-set*). The third component of a fact-type declaration determines whether a restriction **Var** or **Function** is applied. For fact-types with the **Var** restriction, only one instance can hold true at a time, so other instances are automatically terminated when an instance is created. For fact-types with the **Function** restriction, instances are automatically terminated to ensure the binary relation defined by the fact-type is a partial function. If a domain δ is omitted for a fact-type declaration, the domain is implicitly the empty tuple, used for fact-types declared via the **Bool** keyword.

Each such fact-type defines a nullary relation. Effectively, these are Boolean variables, as their only instance receives a Boolean truth assignment in the runtime state. Fact-types defined with **Fact** and without **Identified by** have the domain **strings** by default.

Duty-, action-, and event-types are associated with a domain of tuples with the fields $x_1, \dots, x_n$ given as the last n arguments to *ddecl*, *adecl*, and *edecl* respectively. In the case of a duty-type, n is ≥ 2 and x_1 refers to the type identifying the *holder* of the duty and x_2 refers to the type identifying the *claimant* of the duty. A duty-type definition thus directly establishes duty-claim relationships. Fact-types are used to model actors in a system; the abstract syntax does not distinguish actor-types from fact-types.

In the case of an action-type, n is ≥ 1 and x_1 refers to the type identifying the actors that can perform the action. Any power-liability relationships introduced by the type are derived indirectly by identifying the actors potentially affected by the postconditions. Note that in the concrete syntax, the actor field is optional and defaults to **actor** if not explicitly mentioned. For example, in the declaration of **start-bidding** the actor field is implicit, whereas in **place-bid** it is explicated as **bidder**. An optional **Recipient** clause is also available but is considered deprecated at present. The Boolean argument to *adecl* determines whether the declaration defines a physical or institutional action-type. A physical action holds true by default as if the action-type has an implicit clauses **Derived from True**. For an example, see the definition of **raise-hand** in Section 3. An event-type is a physical action-type without an associated actor. Physical actions and events do not raise violations but may synchronise with institutional actions that do. A type defined by an earlier declaration within a specification can be re-defined/overridden by a later declaration within a specification.

A type extension adds clauses to an existing type. Which clauses can be added to a type depends on the kind of type. All types can have preconditions (*cond-by*) and derivation rules (*deriv-from*). Note that **Holds when** is syntactic sugar for **Derived from** and does not occur in the abstract syntax. All types can have open and closed clauses. Only duty-types can have violation conditions (e.g., **payment-duty**). Only action- and event-types can have postconditions and synchronisation clauses (e.g., **raise-hand**). Only fact-types can have domain clauses.

All clauses are ‘accumulating’ in the sense that zero, one or more clauses of the same kind can be associated with a type. Each kind of clauses has a specific resolution strategy to handle the absence or multiplicity of clauses of this kind. Preconditions are conjunctive; all need to be true for an instance of a type to be enabled. Derivation rules are disjunctive; all instances of a type computed from derivation rules of that type are considered to hold. A type is considered closed unless there are only *open*() and no *close*() clauses associated with the type. A type is thus closed by default, can be opened once, and can subsequently be closed forever. Violation conditions are disjunctive; only

one suffices for a duty to be violated. Domain clauses are used to either (a) add new string or integer literals to the already finite domain of a fact-type or (b) reduce the infinite domain of a fact-type to a finite enumeration. For example, the following type extension enumerates possible prices of the previously infinite type `price`.

`Extend Fact price Domain 100, 200, 300, 400, 500`

Semantically, type declarations and extensions define a transition system. A state in this transition system is a finite set of truth-assignment (true or false) to instances of the types. Well-typed instances not mentioned in the state are implicitly ‘unknown’. Tested but unknown assignments default to false for closed types or to a runtime exception for open types (further described in Section 5.2.2). As such, the set of possible states of the transition system is determined by the domains of the types. The postconditions and synchronisation clauses of action- and event-types determine the transitions of the transition system by modifying the truth-assignments of the current state to obtain a next state. A transition is labelled with the synchronised actions and events that together manifest the effects required to realise the transition between the two states. A *trace* is a sequence of transitions from some starting state.

The effects of postconditions and synchronisation clauses manifest themselves in parallel: when an action *a* is triggered, all instances computed from the expressions of the postconditions of *a* are created/terminated and all actions with which *a* synchronises are triggered. In this way, a triggered action can cause a chain-reaction of effects resolved synchronously. The *obfuscate*(*t*) postcondition is a special case of *terminates*(*t*) that removes any truth-assignments to the instances computed from *t* rather than assigning them false. This variant is needed when working with open types. The effects of actions and events include possible violations: a triggered institutional action is considered violating if it was not enabled at the moment it was triggered.

The subset of the language with only type definitions, postconditions and synchronisation clauses can be considered the imperative subset of the language, useful for describing process models. Orthogonally, the subset of the language with only derivation rules can be considered the logic programming subset of the language, useful for inferring new knowledge from existing knowledge. Finally, the subset of the language that adds violation conditions to duties and preconditions to institutional actions is the normative part of the language. These clauses give normative meaning to specifications by labelling states and transitions as violating respectively.

4.3. Top-Levels and Language Evolution

Table 1 gives an overview of the changes between the most notable versions of eFLINT. The referenced top-level constructs are formally defined in Figure 5.

eFLINT-v1. The original and primary goal of eFLINT has been to assess concrete scenarios for compliance. A

$q \in$	queries	$::=$	$bool\text{-}query(t)$ $ $ $inst\text{-}query(t)$
$sc \in$	stmts	$::=$	$trigger(t) c$
$script \in$	scripts	$::=$	$(sc q)^*$
$spec \in$	specs	$::=$	$(dc ext)^*$
$p \in$	phrases	$::=$	$sc q seq(p_1, p_2)$ $ $ $decl\text{-}block(spec)$ $ $ $phrase\text{-}block(p^*)$
	programs	$=$	$(spec, domcl^*, i^*, script)$

Figure 5: Abstract syntax of eFLINT top-level constructs.

scenario is a sequence of statements and queries. The set of all scenarios is called **scripts** in Figure 5 for consistency with [15]. Statements either trigger actions/events or directly create/terminate/obfuscate instances. Given a specification, a well-typed sequence of statements identifies a trace in the transition system underlying the specification. Queries are expressions evaluated in the context of the current state and either produce a Boolean result (*bool-query*) or a collection of instances (*inst-query*).

Rather than a specification and a scenario, eFLINT-v1 accepted **programs** as defined in Figure 5 (modulo the changes to other syntactic sorts made since). In addition, a program admits a sequence of domain-related clauses (**dom-cls**) to make finite the domains of some or all types of the specification. Moreover, a program contains a sequence of instances to explicitly introduce an initial state, considering all the instances mentioned in the sequence as initially holding true. This specific separation was motivated by a one-to-many relationship between the different components: the same specification can be refined in multiple ways, with multiple initial states defined and multiple scenarios executed. The interpreter for eFLINT-v1 could also be used for simulation by loading the interpreter with a program and stepping through a scenario manually.

eFLINT-v2 and -v3. Where programs are useful to assess the compliance of hypothetical and historical scenarios, something more flexible was needed to assess the compliance of dynamically evolving scenarios at runtime. Programs are especially limited in situations where the specification of the scenario is evolving. Following the theory of [19], the language was redesigned with eFLINT-v2 to enable the incremental development of a specification and scenario through the execution of *phrases* (see Figure 5). Phrases enable declarations and statements to be mixed arbitrarily. This version also introduced the notion of type extensions. In eFLINT-v1, declarations had a fixed set and amount of clauses per kind of type. Since eFLINT-v2, a type can accumulate clauses via type extensions.

The flexibility of the phrases simplified the implementation of reasoner APIs on top of the interpreter. For example, in [91], an architecture is presented for normative reasoning in actor-oriented programming, enabling actors

Table 1: An overview of the evolution of eFLINT. The versions v1 to v4 are referring to different versions of the reference interpreter. Version eFLINT-clingo is an alternative implementation translating eFLINT to Clingo. Row ‘Top-level’ is referring to a syntactic sort in Figure 5.

	eFLINT-v1	eFLINT-v2	eFLINT-v3	eFLINT-v4	eFLINT-clingo
Year	2019	2021	2023	2026	2025
Citation	[15]	[14]	–	This work	[42]
Top-level	programs	phrases	phrases	phrases	specs × scripts
New		Phrases Type extensions Synchs with	Instance queries Open and Physical Conditioned by	Phrase blocks	Negation as failure Search
Removed / Omitted / Deprecated		Recipient	Domain constraints Force action effects	Dynamic Holds When	Simplified Enabled Open and Physical

that modify their interpretation of norms. The application of eFLINT reasoners triggered various language and implementation changes in eFLINT-v3 such as instance queries and open types whose values are determined by the environment rather than the specification.

eFLINT-v4. Officially introduced with this paper, eFLINT-v4 can be seen as a consolidation effort in which the syntax and semantics were modified for consistency. For example, relative to eFLINT-v3, the semantics of **Physical** and **Open** were updated according to their definition in this paper. The only breaking change introduced by eFLINT-v4 is the modification to the semantics of **Holds when** to ensure that states are finite, resulting in the current implementation that **Holds when** is syntactic sugar for **Derived from**. This change is further discussed in Section 5.

In eFLINT-v4, the top-level syntax is a sequence of phrases that can include *blocks* of phrases. Semantically, phrases are executed in sequence, with each encountered block having its phrases executed in parallel as one step in the underlying trace. The execution of a block first distils and processes all the declarations in the block, such that new types are in scope, and then executes all the statements and queries in parallel.

eFLINT-clingo. Version eFLINT-clingo was developed for two main reasons: resolving the semantics of negation as failure and enabling search through answer-set programming. As is discussed extensively in [42], the reference interpreter does not properly handle negation in some specific situations, resulting in conclusions that are inconsistent with the derivation rules. The implementation of eFLINT-clingo is an interpreter that internally compiles a specification and a scenario to Clingo. The Clingo solvers is used for analysing the trace identified by the scenario, e.g., to see which instances are enabled in certain states and which violations occurred. The output of the Clingo solver is translated back to the user of eFLINT.

Leveraging the abilities of Clingo, it is possible to search for traces that satisfy criteria. For example, with a few lines of Clingo added to the generated output, it is possible to find all action-compliant traces that result in duty-

violations. In future work we aim to add additional top-level syntax to eFLINT-clingo for expressing such search tasks and model checking properties directly in eFLINT syntax. Section 6.2 provides details on the features of eFLINT-v4 not yet present in eFLINT-clingo.

As is analysed in Section 6.3, eFLINT-clingo is the most performant in almost all situations with respect to runtime. The variant also received industrial uptake; eFLINT-clingo was chosen by industry partner Abykys for the operationalisation and utilisation of eFLINT to govern data sharing transactions. Section 6.4 provides more details and a runtime performance experiment based on an Abykys use case.

The evolution of the language was driven by additional requirements resulting from various applications of the language. Section 5 reflects on specific design decisions that were made in response to these requirements. Section 7 reflects on the applications and discusses the extent to which these requirements are met.

5. Design Reflections

This section reflects on the most important design decisions made in order to better meet requirements identified across eFLINT’s evolution. The requirements are described in Tables 2 and 3 and are referred to in the text in **bold**.

5.1. Knowledge Representation

Types. The expressiveness of eFLINT’s knowledge representation can be demonstrated in connection to relational algebra. Relations in eFLINT are algebraic products, akin to database tables, predicates in Prolog [32], or triples in RDF [75]. A multitude of derivation rules associated with a type naturally captures the concept of union from relational algebra. Joins can be achieved by applying **Exists**, **Forall** and **Foreach** operators to enumerate relations and using **When** to constrain the result. The combination of accumulator operators and **When** enables grouping.

In many programming languages users rely on algebraic sums – the dual of products – to represent the concept of

Table 2: A description of legal and reasoning requirements.

<i>legal</i>	The language supports ...
interpretation	the formal interpretation of a wide range of norms, e.g., laws, regulations, and agreements
qualification	the formal qualification of real-world events and objects to institutional actions and facts
normative concepts	the most fundamental normative concepts, e.g., permission, obligation, power, etc.
abstraction level	formalising normative sources at their own level of abstraction
open-texture terms	terms whose meaning is left abstract for context-specific specialisation
specialisation	redefining types to provide a specialised definition with a specific context
assessment	assessing the compliance of scenarios, e.g., legal or social cases
ex-ante enforcement	assessing the compliance of actions (processes) before they are performed (executed)
ex-post enforcement	formalising responses to violations such as penalties and compensation
auditability	recording the information required to establish how compliance decisions were made
<i>reasoning</i>	The language supports ...
forward chaining	applying derivation rules and transitions in ‘forward’ direction
backward chaining	applying derivation rules and transitions in ‘backward’ direction
norm priorities	defining strategies to resolve conflicts between norms
infinite domains	reasoning without enumerating all potentially relevant instances a priori
finite domains	reasoning with a finite set of potentially relevant instances
reducible domains	modifying the set of valid instances of a domain, admitting/simplifying finite reasoning
open world	reasoning with unknown truth-values
closed world	reasoning with the closed world assumption
micro-simulation	playing out hypothetical scenarios to see the effects of actions and their alternatives
macro-simulation	simulating the effects of interpretations on sets of scenarios to anticipate implementation
search	finding scenarios that satisfy certain (un)desirable properties

heterogeneous collections, e.g., a set of mixed strings and integers (**datatypes**, Table 3). Like in Alloy [69], heterogeneity is expressed by using the same facts in relations of different types. For example, `citizen-of(person("Amy"), country("Germany"))` and `member-of(person("Amy"), club("Chess", elo(500)))` treat persons as heterogeneous collections mixing memberships and citizenships.

Act-types and duty-types behave as (relational) fact-types in how instances are represented, constructed, created and terminated. This design choice has made it simple to represent the legal concept of power. To change the normative positions of (other) actors, an actor can be given actions that create or terminate facts, actions or duties that affect that actor (see **normative concepts** in Table 2).

Enumerating Domains. When using eFLINT for runtime enforcement, the norm specification is likely to require types with **infinite domains** (Table 2). After all, the instances involved in a scenario may only become apparent as the scenario unfolds. A pragmatic design decision has been made to give semantics to the **Foreach**, **Forall**, and **Exists** operators depending on the domains of the types they enumerate. The instances of a finite type are all enumerated, independent of whether they hold true or not. Of an infinite type, only instances that hold in the current knowledge base are enumerated. By design, enumeration will therefore always terminate. In the former case, termination is guaranteed because the domain is finite and only finitely many instances exist. In the latter case, termination is guaranteed because knowledge bases are finite by design. The difference is subtle, impactful, and may

result in surprising, difficult to diagnose specification errors. After all, knowledge of the domain of the type is required to predict the enumeration, which is tricky because the domain of a type might be changed by a later extension of the specification.

As an example, the value of **count-all** in the following fragment differs depending on whether the first or second definition of **numbers** is used. With the first definition of **number**, all numbers 1 to 5 will be counted. With the second definition, only numbers 1, 3, and 5 will be counted because **Int** identifies the infinite domain of all integers and because only 1, 3, and 5 are in the knowledge base.

```
Fact number Identified by 1..5. //finite domain
Fact number Identified by Int. //infinite domain
+number(1). +number(3). +number(5).
Var count-all Identified by Int Derived from
    Count (Foreach number: number).
Var count Identified by Int Derived from
    Count (Foreach number: number When Holds(number)).
```

The definition **count** has the **When** clause to ensure only numbers that hold are counted irrespective of whether **number** was defined with an infinite domain. This construction can be used to ensure that the enumeration of a type does not change if the domain of the type is modified.

Knowledge bases in eFLINT are finite because instances of types are concrete rather than symbolic, i.e., instances are fully instantiated and do not contain placeholders. The statement **+rich(person)** in the following code fragment determines that every *currently* known person is rich rather than every conceivable, earlier known, or later known person. As a consequence, Chloe is not considered rich at any moment in the execution of:

Table 3: A description of requirements related to services and usability.

<i>services</i>	The language supports ...
case input	receiving information from the external environment as input
case output	providing information to the external environment as output
service interface	specifying which information can be provided as input to a specification
persistence	persistent storage of all runtime state needed for reasoning
dynamic updates	modifying the specification at runtime
data migrations	modifying instances to recover their validity w.r.t. a modified type
event-based	responding to events, actions and observations about situations in the environment
instantiation	instantiating specifications within specific contexts, e.g., via a parameter mechanism
<i>usability</i>	The language supports ...
datatypes	primitive, composite and user-defined types to represent facts about the world
domain-users	both legal experts and software experts in their purposes for using the language
source references	connecting code fragments to fragments from the normative source such as a regulation
transparency	communicating about interpretation and compliance decisions to affected stakeholders
explainability	explaining compliance decisions to relevant stakeholders
versioning	maintaining multiple versions of specifications and scenarios
verification	verifying a specification through testing, debugging, and/or property checking
modularity	establishing specifications and scenarios modularly
extensible	extending existing specifications in a modular fashion, e.g., for specialisation and exceptions

```
Fact person.
+person(Alice). +person(Bob).
+rich(person). +person(Chloe).
```

The statement `+rich(person)` is to be interpreted as the imperative “for each known person, assert they are rich”, rather than the declarative “every person is rich”. (This statement has an implicit `foreach`, explicated as part of name resolution.) Note that in applications in which the set of known persons does not change, the declarative and imperative interpretation are consistent. The decision to maintain concrete facts has been made on the assumption that such knowledge bases are easier to understand for legal experts (**explainability** and **domain-users**, see Table 3). The declarative interpretation can be formulated using a derivation rule to ensure every known person is considered rich, in every knowledge base:

```
Fact rich Derived from person
```

The semantics of enumeration also affects derivation rules. Consider the action `start-bidding` of the running example. In the following code fragment, the three derivation clauses associated with the action are equivalent. The first can be seen as syntactic sugar for the second and the second is fully explicated by the third.

```
Act start-bidding Related to object // implicit actor
Holds when auctioneer(actor)
Derived from
start-bidding(actor, object) When auctioneer(actor)
Derived from (Foreach actor, object:
start-bidding(actor, object) When auctioneer(actor))
```

The query `?Holds(start-bidding(David, Vase))` does not hold as `object(Vase)` is not in the knowledge base. That `object` affects the first clause is counter-intuitive as line 2 does not mention `object`. In an alternative semantics, we considered binding the field names of the type to the fields of a given instance when determining whether

that instance holds true according to a `Holds when` clause. In the example, `actor` is then bound to `David` and `object` to `Vase`. In this case, `object` is not enumerated as the variable is already bound and the query holds as `David` is an auctioneer. The alternative semantics conflicts with the previous design choice that knowledge bases are concrete and finite. In the alternative, `start-bidding(David, object)` would hold true for all conceivable objects. The alternative semantics has been considered and implemented, but has been discarded with eFLINT-v4.

Compared to other logic programming languages, the strategy followed by eFLINT for enumerating the values of types with infinite domains is most comparable to the default strategy for grounding rules in Datalog. In Datalog, the possible instantiations considered for variables are determined by the set of true facts of a program. This is also the case for the unbounded types of eFLINT. Note, however, that the current set of known facts can dynamically change in eFLINT, e.g., following the execution of actions and events. In contrast, eFLINT enumerates all known facts for bounded types, not just those that hold. Clingo applies a grounding phase to programs, grounding each rule by replacing the rule’s variables with all possible instantiations of the variables. The domain from which these instances are chosen is determined syntactically, e.g., by including the atoms that occur in rule bodies and heads. Datalog, Clingo and eFLINT have in common that they reason ‘bottom-up’ by computing a fixed point from known facts. Prolog, as a ‘top-down’ language, reasons from a goal by applying rules in reverse and by using a unification algorithm, thereby avoiding the need to enumerate domains. The programmer needs to be mindful when writing rules in order to avoid non-termination caused by, for example, left-recursion or (other) problematic cyclic rules.

Restricting Domains. To dynamically restrict the domain of a type, eFLINT allowed writing **When** <bool-expr> after an **Identified by** or **Related to** clause. The following (abbreviated) example taken from [15] determines that all valid instances of **collect-personal-data** must satisfy the condition **subject-of(subject,data)**.

```
Act collect-personal-data
  Actor controller Recipient subject
  Related to data, processor, purpose
  When subject-of(subject, data)
  ... // other clauses have been omitted
```

Combinations of **controller**, **subject**, **data**, **processor**, and **purpose** for which the constraint does not hold are thus not members of the type **collect-personal-data** and are therefore not enumerated (e.g., by **Foreach** or **Forall**). This feature was considered crucial in eFLINT-v1 which enabled simulating scenarios (**micro-simulation**, Table 2). The interpreter can present (also in eFLINT-v4) users with a finite list of triggerable actions and events. For each, the interpreter shows whether the action or event is permitted in the current state, thereby showing the difference between *ability* and *permission*. The feature to restrict domains served to constrain physical ability separate from compliant behaviour. Note that the feature essentially gave eFLINT a data-dependent type system, complicating its semantics. For these reasons, the feature is considered deprecated since eFLINT-v3. As an alternative, applications can benefit from the **Physical** keyword and present only enabled physical actions to users. The domains of types can also be statically reduced by extending a type with a **Domain** clause (**reducible-domains**, Table 2).

The desired separation between *permission* and *power* motivated the design change in eFLINT-v3 that actions always manifest their effects when executed, even when not permitted. The keyword to **Force** the effects of actions, was deprecated by this change. The construct **... When Enabled(...)** can be used to ensure effects manifest only when an action is permitted.

Implicit Arguments. In a constructor application with implicit arguments, any missing arguments are implicitly the name of the missing field (see Section 4.1). Implicit arguments simplify the translation from FLINT to eFLINT. Consider the FLINT act-frame on the top of Figure 1 and the corresponding eFLINT code on the bottom. The eFLINT code can be automatically generated from the FLINT frame directly. The resulting eFLINT code is internally consistent and executable. However, there are some issues with the code. For example, multiple (pairs of) spouses can be represented as instances of the type **[spouses]** but only one valid marriage can be represented as **[valid marriage]** is a Boolean type. This problem can be fixed in at least two ways by redefining some of the types (**specialisation**, see Table 2). Firstly, **[spouses]** can be defined using the **Var** keyword, encoding an assumption that reasoning involves at most one pair of spouses. Secondly, **[valid marriage]** (and **[marriage attempt]**)

can be re-defined and given a field **[spouses]** to determine which pair of spouses are married:

```
Fact [valid marriage] Identified by [spouses]
```

The original act-type definition is still valid with either solution, owing to constructor application with implicit arguments. In the second solution, the **Creates** expression is equivalent to **[valid marriage]([spouses])**. When an action is performed and the expression is evaluated, **[spouses]** is bound to the recipient of the action. The support for implicit arguments and quantifiers in eFLINT thus ensures the direct translation from FLINT to eFLINT yields executable code although extensions may be required to obtain the desired semantics specific to the use case.

The decision which of the two solutions to apply is influenced by the design of the software system in which the code is applied. If the code is applied to reason about individual, historical cases, then the first solution is effective. If the code is integrated in a system in which multiple cases are managed, then the second solution might be preferred. An advantage of the design of eFLINT is that both solutions can co-exist: the original definition in Figure 1 can be extended by two separate files (using the **#require** directive), each implementing one of the solutions. A third solution is to run a dedicated (process within an) eFLINT reasoner for each individual case, **instantiating** (Table 3) the preferred extension with the correct spouses.

5.2. Service Interactions

For dynamic regulatory compliance checking of software (defined in Section 2.2), some process is required to automatically convert system events into the steps of a scenario (the **case input** requirement, Table 3). Conversely, the effects of the statements in a scenario need to be observable and interpretable by the system, e.g., to respond to violations (**case output**, Table 3). Initially, a strict separation was maintained between physical processes (external to eFLINT) and institutional processes (internal to eFLINT), relying on code external to eFLINT to make the connection. However, experience demonstrated that it is convenient to use **Physical**, **Syncs with** and derivation rules for expressing the required **qualification** (Table 2) within eFLINT. Doing so, eFLINT actions and events model both physical and institutional process, and the two are explicitly connected by eFLINT clauses.

5.2.1. Incremental and Exploratory Programming

The desire to assess dynamically evolving scenarios and interpretations, motivated one of the most impactful design decisions: with eFLINT-v2, the incremental programming style of languages such as Python was adopted. In this style, a program is a sequence of individually valid program fragments, referred to phrases in Section 4.3 and [19]. This is a natural fit with **event-based** applications and enables **case input** to be given as phrases to the interpreter, and its immediate feedback to be emitted as **case output**. For example, the “raise hand” button-event can be converted

to an eFLINT phrase triggering the corresponding action. When the auctioneer ends the bidding, the resulting duty to pay can be emitted as a notification to the highest bidder. The incremental style of programming makes specifications **extensible**, **modular** and enables **dynamic updates** and **specialisation**.

The change was realised by applying the design methodology presented in [19] and implemented using the generic interpreter back-end of [46]. Benefitting from this back-end, eFLINT supports exploratory programming, which enables revisiting previous runtime states for debugging (see **verification**) and **micro-simulation**. The execution history contains information to retrace causes and effects, enhancing **explainability**, **transparency** (Table 2) and **auditability** (Table 3).

For security reasons and to separate concerns, it is beneficial to restrict an eFLINT reasoner to only respond to certain inputs and to make other types strictly internal. This way, for example, we can disallow any duties from being created or terminated directly, but instead only through institutional actions (powers). Similarly, we can restrict the creation of authorizations to specific actors and thereby prevent actors from granting authorizations to themselves. For the running example we might like to say that only the **raise-hand**, **start-bidding** and **end-bidding** actions can be triggered. A possibility is to extend the language with a module-system. At present, we consider that an eFLINT specification implicitly describes a **service interface** (Table 3) by exposing only physical actions, events, and open types (see below). An application-specific eFLINT reasoner can impose further restrictions on the available interactions.

5.2.2. Open Types

The **Open** and **Closed** modifiers are available to indicate whether the **closed world** or **open world** (Table 2) assumption holds for the declared type. The logic for open types is three-valued: true, false or unknown. An instance is true or false if and only if it has been explicitly assigned a truth value as part of additional input (described below), or using **Creates**, **Terminates**, **+<EXPR>**, **-<EXPR>**, or when it is derived as true. The truth of all other instances is unknown. A closed type can be seen as a special case of an open type for which unknown is also interpreted as false. The **Obfuscates** clause has been added to the language to remove the truth assignment of an instance.

The open types of a specification serve as parameters of the specification capturing knowledge only available within the environment in which the specification is applied (**service interface**). As a rule of thumb, one can declare those (fact-)types as open whose truth values are not modified by the specification itself. For the running example, we could decide the following:

```
Open Function min-price-of Identified by object * price
Open Fact bidder Identified by String
Open Var auctioneer Identified by String
```

Note that the information captured by open types may be subject to change during execution. For example, it may be reasonable to assume additional bidders are added, whereas perhaps the minimum price of an object is fixed once set. A mechanism has been added to eFLINT-v3 to provide ‘additional input’ alongside a phrase to instantiate certain types for the evaluation of this phrase only. For establishing truth, input has a higher priority than creation/termination which has a higher priority than derivation.

The treatment of unknown affects how applications interact with eFLINT reasoners. The main design decisions are what happens when (a) evaluation requires the determination whether an unknown instance holds true and when (b) evaluation requires the enumeration of the instances of an open type with an infinite domain. The execution of a phrase is interrupted, and an exception raised, when an unknown truth-assignment to an instance is required to evaluate a Boolean expression. The execution of a phrase is also interrupted when the instances of an open type need to be enumerated and the domain of the type is infinite. For open types with an infinite domain, the truth of all instances not recorded in the knowledge base is unknown and it is therefore unknown which instances require enumeration. In this case, the open type itself is reported as part of an exception alongside the interruption.

Both types of exceptions can be understood as a request to the execution context to provide additional input. The first kind of exception can be resolved by the execution context providing a truth assignment to the reported instance. The second kind of exception can be resolved by the execution context providing the instances within the domain of the reported type that are to be enumerated. To resolve exceptions and continue, additional input is provided as a set of truth assignments to instances.

Two motivations for closing a type previously declared as open have been encountered. Firstly, certain static analyses may require finite domains and may require the closed world assumption for all types. Secondly, a specification with open types may be extended to specialise towards a particular application. For example, an open-type such as **bidder** may be refined to include knowledge about valid bidders in the specification. Within a particular application it may be valid to say that “all users except the auctioneer are valid bidders”, expressed as follows:

```
Open Fact user Identified by String
Closed Fact bidder Identified by String
Derived from user When Not(auctioneer(user))
```

The newly introduced type is typically an open type, such as **user** in the example above. With these definitions, **bidder** can be considered ‘internal’ and **user** ‘external’.

5.3. Reasoning by Default

The type of reasoning applied to determine the truth assignment of instances is known as *reasoning by default*: instances hold if they were created or derived by some clause, and otherwise, by default, they do not hold. This semantics imposes an asymmetry between truth (holding)

and falsity (not holding): only truths are witnessed, by creations or derivations. How can we be certain that at any given moment, all truths have been witnessed to definitively evaluate occurrences of `Not`, `Foreach`, `Count`, etc.?

The literature on logic and logic programming extensively explores reasoning by default (also called *negation as failure*) [29]. Its definition of falsity as the complement of truth is simple and powerful. But, as with many logic programming languages before, reasoning in eFLINT is complicated by the existence of syntactically valid specifications which have no single logical interpretation. For example, which bidders are ready, according to the definition below?

```
Fact ready Identified by bidder
Holds when Not(ready(bidder)).
```

In general, the issue arises from any case where truth is cyclically dependent on its own falsity. These dependencies can be more subtle than in the example above. In the example below, the contradiction of defining minimum prices as being lower than themselves is not as obvious. This issue can be corrected by replacing `<=` with `<`, altering its meaning to specify that no (different) assignments of value to the same object may be larger.

```
Function min-price-of Identified by object * price
Derived from bid Where
Not(Exists min-price-of':
  min-price-of'.object == bid.object
  && min-price-of'.price <= bid.price).
```

Specifications without these troublesome cyclic dependencies are called *stratified* because, intuitively, facts can be categorised into ordered strata, such that the facts depend only on facts in preceding strata, necessarily having no cycles. Several approximations of this ideal notion of ‘stratification’ have been explored [97], because it is generally costly to accurately recognise and exploit.

The original work on eFLINT did not consider the subtleties of reasoning by default. For many years, the interpreter implemented a ‘greedy’ algorithm, which escaped any cycles by overriding falsity with truth as necessary. This solution sufficed to enable years of experimentation and application. Many problems were naturally expressed by stratified specifications. Consequently, the interpreter usually behaved as expected. Otherwise, issues were usually discovered quickly, and could be worked around by just enough refactoring until the underlying cycles vanished. But the reasoning algorithm was found to exhibit another, even more subtle undesirable characteristic: users could not predict how the algorithm breaks the tie between mutually exclusive truths [42]. For example, nothing in the following specification helps the reader to predict exactly which bidder becomes the only auctioneer by default.

```
Extend Fact auctioneer Derived from bidder
When Not(Exists auctioneer': auctioneer' != bidder).
```

As with cyclic dependencies, minor refactoring usually suffices to break all ties. However, in complex specifications, it can be very difficult to recognise the problem and to diagnose the tie or cycle that is to blame.

In [42], we adapted the *answer set* semantics (also called the *stable-model* semantics) [51] for eFLINT. Intuitively, it enumerates all possible outcomes from all acyclic interleavings of reasoning steps. Stratified specifications still have unique answers but unstratified specifications can have no answer - in the case of cycles - or many distinct answers - in the case of ties. Either case recognises the presence of problems in the specification, which users can then address.

In an on-going, separate research effort we are exploring the *well-founded semantics* [112] as an alternative to the answer-set semantics (e.g., see the related Seaso language in [39]). These semantics are closely related, but they differ in a crucial detail: like eFLINT’s original ‘greedy’ semantics, the well-founded semantics ensures that every specification has a unique answer, and like the answer-set semantics, the truth assignment of the answer preserve the constraints imposed by derivation rules. The trick is that the well-founded semantics assigns unknown as a distinct value to instances, and does so to those instances whose truth or falsity would be inconsistent with the constraints. The well-founded semantics thus marks and isolates troublesome instances for users to diagnose. In the prior **auctioneer** example, the set of bidders remains clear and determined, but whether each bidder is an auctioneer is unspecified, because any Boolean valuation would arbitrarily prefer one instance over another. The well-founded semantics and answer-set semantics align in an intuitive way: truth and falsities in the former have the same value in *all* answers of the latter [112]. That is, the well-founded semantics can be understood to ‘summarise’ the answer-set semantics.

6. Implementations

This section discusses several aspects related to the implementation efforts of the eFLINT language, focussing primarily on the reference interpreter versions 1 through 4 in Haskell and eFLINT-clingo interpreter implemented in Rust. The motivations behind the two implementations are discussed and performance comparisons are made.

6.1. The Reference Interpreter

The Haskell implementation of eFLINT [13] serves as a tool to experiment with the semantics of the language and to prototype with alternatives. The decision was made to implement eFLINT as a definitional interpreter, thus simultaneously defining and implementing the language. The main requirements for the implementation were code readability and that adjustments should be simple to make. The evaluation functions central to the interpreter are implemented via a principled usage of well-established Haskell monads: the Reader monad for dealing with bindings and scope, the State monad for state transitions, and the Writer monad for output and exceptions. The resulting definitions are comparable to a big-step style structural operational semantics and, for the well-acquainted, concisely convey a lot of information about the implemented semantics. The

grammar combinators of [17, 18] are used to define the concrete syntax of the language. The result is a syntax definition that resembles abstract syntax quite closely whilst defining a (GLL) parser directly. These and other implementation choices promoted the requirements, generally doing so at the cost of performance.

For these initial purposes, the interpreter was sufficiently self-documenting. However, as the user-base started to grow, other demands were being placed on the (tool) support for the language. Firstly, additional forms of documentation were added. Jupyter notebooks were developed to give a gentle and high-level introduction of the main language features, curated examples were taken from experiments and use cases, and an ever-growing set of unit tests help testing and documenting expected behaviour of specific language features and corner cases.

Secondly, additional front-ends and means to interact with the interpreter were needed, resulting in REPL and API front-ends replacing the initial simulator front-end. The REPL front-end replaces the simulator in the most direct sense and on top of the usual REPL interactions offers meta-commands for exploring (alternative) scenarios in a step-by-step fashion. The API front-end has been used to develop several wrappers for accessing the interpreter as a reasoner, e.g., in Python, Java, and Scala for actor-oriented programming within the AKKA framework. Especially for users of the API front-end (and the users of the software they build), the runtime performance of the interpreter renders it impractical beyond basic examples. For other users, the interpreter serves as a reference implementation to verify their own experimental implementation of the language. For this reason, we decided to maintain the original requirements for the Haskell interpreter and investigate alternative, more performant implementations.

Thirdly, with more users, eFLINT was exposed to additional scrutiny and over time it was shown that the documentation was not always complete. Specifically, the Jupyter notebooks and curated examples are too high-level for those seeking a detailed understanding of the language. The test suite provides further details but was repeatedly found to be incomplete and had to be extended as new bugs or corner-cases were discovered. Most problematically, in some cases the interpreter was discovered to exhibit behaviour that was not expected or desired upon closer inspection. These cases occurred when implementation choices in the interpreter were not made consciously but rather followed naturally from the way the code was organised or other features were implemented. This prompted the effort to develop a formal semantics of the language.

6.2. The eFLINT-clingo Interpreter

From the observations of the previous subsection we concluded that a formal definition of the semantics of the language is needed, triggering the work on a translation from eFLINT to the Clingo language reported in [42]. Specifically, this translation and resulting implementation addresses the following points at once:

- (a) The eFLINT semantics is formally specified to fully document the language’s main features in all their details. This point is addressed by specifying a translation from eFLINT to (a subset of) Clingo, inheriting from Clingo’s semantics in several regards.
- (b) The translation addresses the Haskell interpreter’s problems with reasoning by default, opting for the stable model semantics of Clingo.
- (c) The runtime performance of eFLINT code is significantly improved, especially in regards to the evaluation of derivation rules, benefitting from the years of development and optimisation efforts that went into the Clingo solver. Clingo’s superior performance has been demonstrated at international competitions [24].
- (d) The implementation supports model-checking and searching for scenarios satisfying given criteria (supporting **verification**, Table 3 and **search**, Table 2).

The implementation consists of the translation pipeline and calls the Clingo solver internally before converting its output back to eFLINT. This has enabled the implementation of a reasoner on top of the eFLINT-clingo interpreter, roughly comparable to the reasoners developed on top of the reference interpreter. However, at present, eFLINT-clingo does not support all reasoning features.

Unimplemented Features. In Section 5 it was discussed that the reference interpreter adopted incremental programming features, supporting its use in cases where user input is interleaved with reasoning and output. Firstly, it raises exceptions to request additional input when evaluating instances of **Open** types. Secondly, it accepts type extensions and statements as phrases.

At present, the eFLINT-clingo interpreter supports neither of these features. A specification and scenario are input first, and evaluated second. Work continues to support these features by re-introducing a layer that maintains a mutable intermediate state. The hope is to exploit Clingo’s more advanced features and configuration interface [49], such that the Clingo solver maintains the intermediate state as much as possible. In the meantime, users can approximate these features by maintaining an specification as mutable state themselves, and interleaving their mutations with calls to the interpreter.

Another semantic difference is that the **Enabled** check on an action a does not involve checking the preconditions of actions that synchronise with a via **Syncs with**. This dramatically simplifies the translation to Clingo and its reasoning about each action in each state. The original semantics is recoverable by (speculatively) performing the action and checking if a violation results.

There is also no distinction yet between **Physical** and institutional actions. We do expect to be able to leverage this distinction for static regulatory compliance checking using the Clingo solver (as defined Section 2.2).

6.3. Synthetic Performance Comparisons

In this section, we report on a suite of synthetic performance tests to compare the reference interpreter for eFLINT-v4 and the eFLINT-clingo interpreter. Recall from Section 5.3 that these two interpreters produce different outputs given some inputs expressing reasoning by default. For a fair comparison on performance, we consider only cases where the two interpreters produce the same output. Evidence of the equivalence of the two implementations in these cases is given in [42]. We initially focus on concise and synthetic inputs crafted to isolate our testing of the performance characteristics of the interpreters to semantic properties of the input. For example, one such property is the length of the input scenario. In Section 6.4 we analyse the performance of the eFLINT-clingo interpreter on a real-world example.

With the crafted tests, we have identified trends in the responses of both interpreters that reveal their suitability to various kinds of input. We find that the eFLINT-clingo’s relative strength is in handling cases of complex, yet structured reasoning in each scenario step, and it is relatively weak in reasoning about long scenarios with trivial reasoning in each state. We ultimately find that eFLINT-clingo generally outperforms eFLINT-v4 by a significant margin, and the few cases of slowdown are modest.

Experimental Setup. Figures 6 to 10 plot the results of an illustrative selection of a larger set of experiments available in full at [86]. Per plot, each measurement is the median of 10 runs on a machine with an Intel core i9 9th generation processor with 16 hardware threads, 32GB RAM DDR4 memory, and an NVMe hard drive. The experiments compare both interpreters’ single-threaded performance. Cursory experiments found that Clingo achieves no speedup from multi-threading for inputs producing just one answer.

Discussion of Experiments. Figure 6 evaluates the performance of the interpreters given an input initialised with many instances, but where only one derivation step is possible at a time. For example, in any scenario, only $x(50)$ is derivable just after $x(51)$ is derived. The interpreters exhibit similar response curves: runtime grows superlinearly with the number of derivations, presumably as a consequence of paying the price of storing and traversing larger knowledge bases. However, eFLINT-clingo starts faster, and slows down more gradually. The net result is that, compared to eFLINT-v4, eFLINT-clingo begins with a $2\times$ speedup and grew to $25\times$ for the largest input that we tested. We have no explanation for $N = 2^{11}$ representing a significant outlier, but we note that it was reliably reproduced by both interpreters.

Figure 7 demonstrated a case where eFLINT-clingo maintained approximately $\times 2$ speedup across all test scenarios. We expect that the cost of the division operator predominates, but more testing is required.

Figure 8 shows the results of an experiment expected to emphasize the strengths of Clingo: the interpreter must

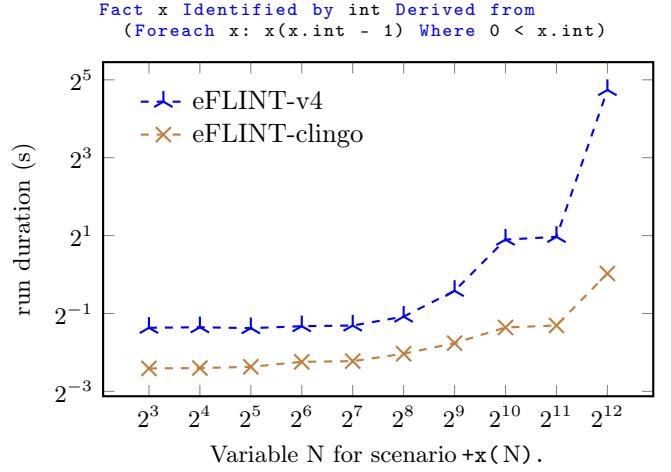


Figure 6: Speed in unfolding long derivation chains.

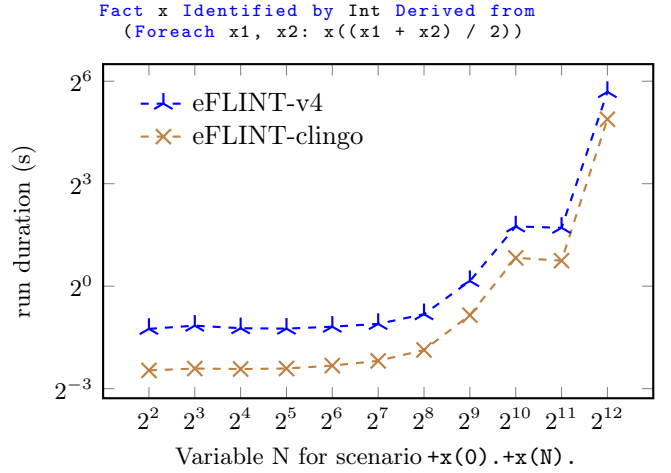


Figure 7: Speed in reasoning with substantial integer arithmetic.

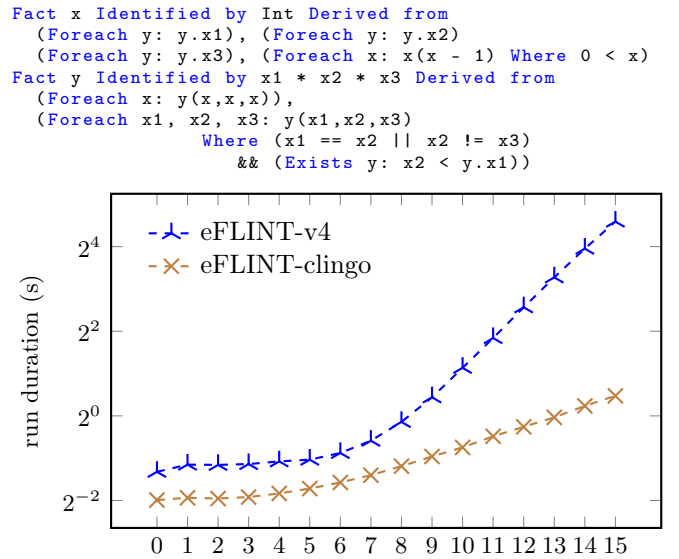


Figure 8: Speed in reasoning in cases requiring the enumeration, combination, and comparison of many instances.

quantify, combine, and compare many terms. This specification causes runtime cost to scale more sharply with the independent variable, but the speedup of the new interpreter over the original are similar to that shown in Figure 6: the simplest scenario induces $2\times$ speedup, and the largest scales up to $17\times$ speedup.

Figure 9 shows a case where the reference interpreter for eFLINT-v4 is expected to excel: minimal reasoning is spread over a long scenario. Intuitively, the reference interpreter was optimised around the ‘chronological’ characteristic of eFLINT scenarios and states: reasoning about each state N depends only on states $N - 1$ and N . The reference interpreter reasons sequentially, wasting no time considering unrelated states. By comparison, Clingo threatens to reason naïvely, because the entire eFLINT specification and scenario are encoded as a monolithic Clingo ruleset, and we cannot assume that Clingo recognises and exploits its subtle chronological property. Indeed, we find that eFLINT-clingo’s speedup peaks with 94% at $N = 2^6$, waning to 82% speedup at $N = 2^{10}$. However, clearly, eFLINT-clingo overall still outperforms eFLINT-v4 by a significant margin, and this experiment suggests that its performance advantage will only be lost in scenarios far longer than we expect to encounter in practice. Nevertheless, we consider this to be an interesting direction of future research, to which we hope to leverage Clingo’s *multi-shot solving* [49], where Clingo’s processing of input is interleaved with reasoning and output.

Finally, Figure 10 shows a case where eFLINT-v4 outperformed eFLINT-clingo. This input is a carefully crafted encoding of the *prime sieving problem*: enumerate all prime numbers to a given maximum [11]. Here, we expect Clingo’s heuristics for selecting rules to fail. Both interpreters use a similar solution: naïvely enumerate triples (X, Y) and test if $X \times Y = Z$ for each Z . Performance then depends on the order that the integer pairs are enumerated. Evidently, eFLINT-v4 gets lucky more often. But this outcome was sensitive to the encoding of the problem. For example, chaining `addleq` via `Derived from` instead of `Syncs with` slowed down eFLINT-v4. More experimentation is required to precisely explain the sensitivity to such particulars of the specification. We conclude that eFLINT-clingo benefits from specifications with more structured reasoning, where Clingo’s heuristics try applicable rules more often.

6.4. Realistic Performance Tests

This section is intended to demonstrate that eFLINT is sufficiently expressive – and eFLINT-clingo is sufficiently performant – to solve real-world problems. The demonstration is borrowed from a deployment of eFLINT-clingo in practice. We briefly describe this usage of eFLINT and report that all inputs were evaluated sufficiently quickly.

The Products and Data Exchange (PDX) is a product released in October 2025 as part of the Dutch Metropolitan Innovations (DMI) ecosystem for data-driven innovations managed by the Dutch Ministry of Infrastructure. The

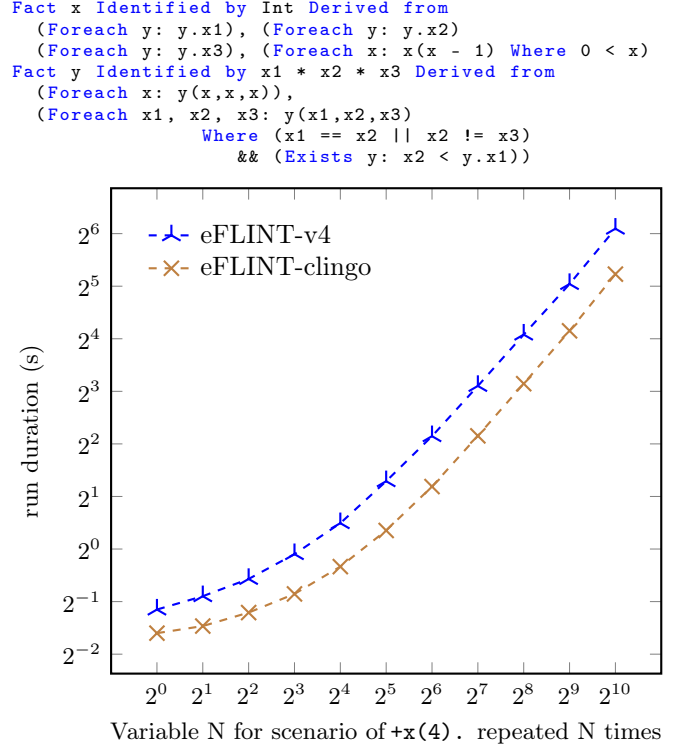


Figure 9: Speed in reasoning about the same, moderate reasoning work at each step of scenarios with varying lengths.

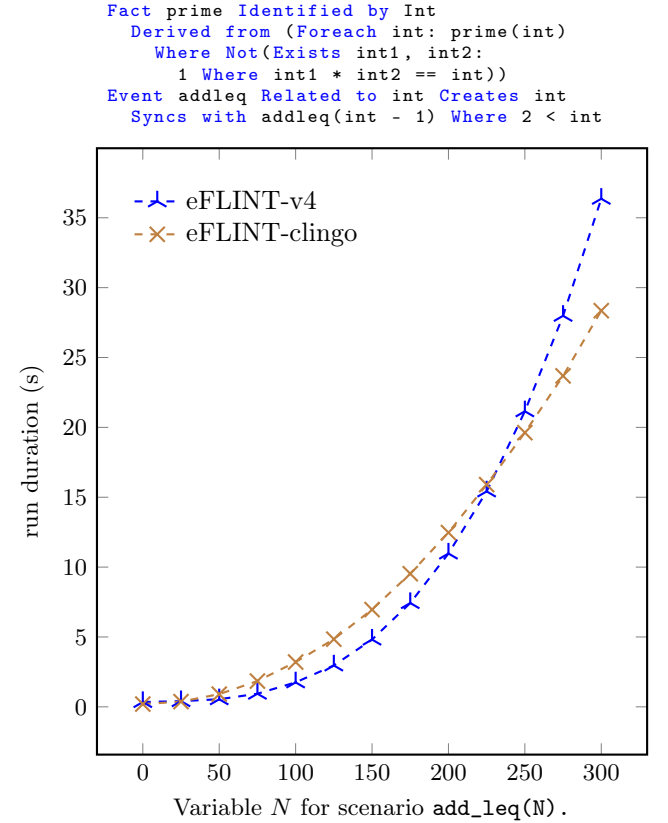


Figure 10: Speed in enumerating all prime numbers up to a maximum.

PDX serves DMI members as a platform for offering, cataloguing, exchanging, and processing datasets and services. A novelty of the PDX is its deep integration and automatic enforcement of regulatory documents, in particular the GDPR privacy regulation and the governance rules of the ecosystem. The governance component of the PDX, developed and maintained by Abykys⁴, is built on top of eFLINT-clingo and enforces the compliance of data-transactions with the regulations formalised in eFLINT.

The ‘DMI Afsprakenstelsel’ defines the rules governing data-transactions and demands compliance with the GDPR. A precise definition of what is automatically enforced is given at the DMI website⁵. The enforcement of the GDPR focusses on purpose limitation, ensuring that data controller and processor agree on a lawful processing purpose.

Experimental setup. The specification of the governance rules is continuously extended, refactored, and refined. We focus on the version that was active on 23 April 2026. The specification consists of 1,543 lines of code, excluding comments and blank lines. With Abykys’ permission, we have included a sample⁶ of their test scenarios, sampling the spectrum from simple to complex scenarios that Abykys encounters. We call these scenarios s_0 (the simplest, consisting of zero statements) to s_3 (the most complex, consisting of 49 statements). Table 4 reports on the time required for (stages of) the eFLINT-clingo interpreter to evaluate each scenario.⁷ These tests were executed on a machine with an AMD Ryzen AI 7 PRO 350 processor with 8 physical kernels and 16 software threads, 32GB RAM DDR4 memory and an NVMe disc. The test inputs are available in their entirety at [102]. The precise integration of eFLINT-clingo into the PDX software is not publicly available; we can only report the assurance of the Abykys developers that their version of eFLINT-clingo remains very close to the version openly available at [86]. The same tests were executed with the reference interpreter for a ball-park comparison of total running times.

The experiments in this and the previous subsection served to compare two implementations of eFLINT. Both implementations deal with the complication that the language is both logical and imperative in nature. Specific applications of the language may only use a subset of the language and may thereby permit more efficient execution strategies. Depending on the features required for an application, choosing an alternative DSL or general-purpose language could therefore lead to better performance than can be realised with the two implementations discussed in

	load	parse	check	trans.	reason	total	v4
s_0	35.97	34.43	13.32	7.55	318.46	409.73	15,087
s_1	37.82	33.78	13.01	7.46	373.40	465.48	15,472
s_2	40.28	36.47	13.65	7.58	1,055.78	1,153.75	15,415
s_3	45.87	42.07	15.07	7.62	3,603.24	3,713.88	19,479

Table 4: Runtime in milliseconds of stages of eFLINT-clingo compared to the eFLINT-v4 total with the *Afsprakenstelsel* as the input specification alongside one of four input scenarios $\{s_0, s_1, s_2, s_3\}$.

Table 5: An list of software using eFLINT. Source code is available by clicking on the software title in a PDF version of this paper.

Title (PoC = Proof-of-Concept)	eFLINT-	Refs
FLINT Rule Editor	v1	-
Medical Data PoC	v2	[14]
Normative Advisors PoC	v2	[91]
Ex-post Enforcement PoC	v2	[81]
eFLINT-check PoC	v2	[34]
Case Management PoC	v3	[107]
Normative Sources PoC	v3	[72]
AMdEX Fieldlab PoC	v3	[16]
Brane Orchestrator	v3	[6, 7, 43]
JustAct PoC	v3	[41]
eFLINT reasoner in DMI	clingo	-

this section. In future work, we aim to work on eFLINT variants that admit more efficient execution strategies, e.g., by giving a fully procedural interpretation to eFLINT. These variants will be more restrictive by giving syntax errors, type errors, or warnings to specifications to which an alternative execution strategy is not applicable.

7. Discussion and Future Work

This section reflects on the satisfaction of the requirements of Tables 2 and 3. References to the requirements are given in **bold**. The reflection is a continuation of the discussion around requirements in Sections 5 and 6 and is based on experience with software systems applying eFLINT. An overview of the software is given in Table 5. After discussing these experiences, Section 7.4 discusses limitations and describes possible avenues of future work.

The development of eFLINT went through phases, starting with the analyses of historical cases and simulation, followed by dynamically evolving cases, model checking and system integration. Applications involving eFLINT have been developed by students, researchers and practitioners at various technological readiness levels (TRLs), with most applications validating concepts ‘in the lab’ ($\leq$ TRL3, following the definition of TRL by the European Committee) or performing demonstrations with stakeholders in realistic test environments ($\leq$ TRL6). Especially notable is the ($\sim$ TRL8) application of eFLINT as part of the DMI ecosystem mentioned in Section 6.4. No applications of eFLINT rely on specific features of a version that make it impossible to migrate to the latest version of the language.

⁴<https://abykys.com>

⁵Here: <https://definities.dmi-ecosysteem.nl> (In Dutch).

⁶Our versions of these test scenarios precisely match the versions made and tested internally by Abykys, except that two strings identifying DMI stakeholders were replaced to protect their privacy.

⁷As can be seen, these cases all spend the majority of time in the ‘reasoning’ stage. But this belies the time Clingo spends actually applying rules, because this work is interleaved with formatting and writing thousands of lines of answer text as output.

7.1. Automating Governance

The eFLINT language has been used in experiments to investigate automating case assessment by governmental organisations based on laws, e.g., for deciding on permits or tax reduction requests. An initial experiment with ICTU – an IT organisation within the Dutch government – developed the first version of an editor for developing norm specifications and scenarios with FLINT and eFLINT. The FLINT Rule Editor was used by ICTU domain experts, successfully confirming the usefulness of the languages for developing formalised **interpretations** of laws which can subsequently be used for automatic case **assessment** and **micro-simulation**. The separation between FLINT and eFLINT in the editor does make it difficult for legal and software experts (**domain-users**) to work together. We expect this limitation can be partially mitigated with better tool support delivering a tighter integration.

Motivated by ‘Rules as Code’ [84], the initial experiment demonstrated how developing machine-readable interpretations of law can promote government **transparency** and can aid resolving disputes caused by differences in interpretation. For these reasons, FLINT establishes **source references** between its frames and the original normative sources in a format following the JuriConnect standard [21]. A later project developed a data structure for storing a multitude of interpretations, efficiently interlinking fragments of eFLINT and normative sources [72], demonstrating eFLINT’s **modularity**, **extensibility** and **versioning**.

In a subsequent experiment, a proof-of-concept (PoC) case management system was developed to assist civil servants in making decisions on cases [107]. The PoC was developed to demonstrate the ability to adapt to changes in laws or policy (**dynamic updates**) and to enforce the powers and duties of both civil servant and citizen. In the PoC, an eFLINT reasoner is **instantiated** to keep track of the progress of gathering information and making decisions on a case. At any time, the system can inform the users of their permitted actions and duties, e.g., to deliver information (citizen) or to make a decision (civil servant). The user interface of the system is populated with buttons and form elements based on information received by the reasoner about the physical actions and open types of the specification as well as the contents of the knowledge base. This mechanism makes the system adaptable in the sense that the UI responds to changes in the underlying eFLINT specification. The PoC thus benefitted from eFLINT’s ability to implicitly specify a **service interface** and to handle **case input** and produce **case output** upon request. The underlying specification naturally considered an **open world** and specified **infinite domains** in order to admit, for example, new citizens registering and submitting unpremeditated numerical information.

Two models were experimented with for keeping track of the progress of cases as they develop over time. The required runtime information is recorded statefully by the interpreter or within a separate database. The former

favours performance as less information needs to be processed for every request. The latter favours resilience as reasoner instances can crash without losing runtime state, thus demonstrating **persistence**. Stateful execution and persistence through an external database are both at odds with adapting to changes in norms, however. After all, changes in norms might require **data migrations** when affecting the fields in type-declarations. At present we have not demonstrated how this can be achieved with eFLINT.

Another potential application of eFLINT to governance is to conduct **macro-simulations** by establishing representative ‘populations’ of hypothetical citizens and analyse sets of cases, such as tax returns, for these populations. This makes it possible to determine the effects of (e.g., tax) policies on subsets of the wider population while the policy is drafted. Although eFLINT should be able to support this application, we have not yet given a demonstration.

7.2. Distributed Systems

In other applications, we have experimented with the automatic enforcement of (privacy) regulations, consortium agreements and data sharing conditions within distributed data exchange systems. In a data exchange system, data producers, consumers and service providers collaborate to execute data processing workflows. We presented a reference architecture for data exchange systems with automatic enforcement of regulations and agreements in [16]. In the AMdEX Fieldlab PoC, implementing the architecture, compliance is checked and enforced dynamically by a reasoner listening and responding to messages, benefitting from the **event-based** nature of eFLINT. An eFLINT reasoner has also been used by the Brane Orchestrator for planning and distributing compliant data processing tasks [43, 6, 7]. The orchestrator can potentially benefit from eFLINT-clingo’s capabilities to **search** for compliant plans rather than relying on the current trial and error approach.

As inter-domain applications, data exchange systems require mechanisms for cross-domain authentication [53] and consensus. Consensus is needed on (a) the applied norms, e.g. to establish which interpretation of the GDPR is applied, on (b) policy input, e.g., whether a particular user is affiliated with a member of a consortium, on (c) the transpired events, e.g., whether data was accessed, and (d) on the decisions made by the normative reasoner, e.g., whether authorisation was given. The JustAct PoC was developed to demonstrate a method to communicate about policies, achieve consensus, and for accountable decision making [40, 41].

In inter-domain applications, **auditability** is of high importance [94, 104]. To achieve a form of accountability, we can translate eFLINT to a smart contract language in a way that the interpretation is recorded as a smart contract and the assessed scenario is recorded on the ledger. We have conducted experiments to confirm the possibility, although a complete, automated translation is not yet available. Evidence of the possibility is provided by [95], presenting a translation from Symboleo [105] to Solidity [44, 115].

We developed multi-agent systems to experiment with different architectures and setups for automating compliance in distributed systems. In these experiments, so-called ‘normative advisors’ are agents that reason with norms based on observations communicated by other agents or by the environment [91]. Our approach made it possible to experiment with different models for distributing institutional facts and institutional reasoning and required multiple eFLINT reasoners to co-exist (**instantiation**). One can define a system with a single agent monitoring and assessing the behaviour of all agents in the system. Alternatively, every agent can have its own normative advisor and use normative reasoning for planning. Such planning activities by agents benefitted from **simulation** of hypothetical scenarios through the **forward chaining** applied by the reference interpreter. The **search** capability of eFLINT-clingo can be used for **backward chaining**, reasoning backwards from a goal state to the current state.

These experiments also included agents for dynamic, **ex-post enforcement** that decide whether and how to respond to observed violations [81]. These agents may be fully autonomous, or may require a ‘human in the loop’.

In [14], an experiment is reported that demonstrate how eFLINT can be used to gradually specialise a generic law to a concrete, actionable set of policy rules. In this Medical Data PoC, elements of the GDPR were concretised by the data processing agreement of a medical consortium. The resulting specification was operationalised by dynamically generating access control policies for governing the access to data, demonstrating **ex-ante enforcement**. The experiment also demonstrated the ability to formalise normative sources as their own **abstraction level**, ensuring that the formalisation of the GDPR fragments can be reused across applications without modification. The formalisation of the agreement demonstrated the **specialisation** of the GDPR provisions to this specific medical context. The same mechanism was used to concretise **open-texture terms**, such as ‘undue delay’ in the GDPR, which are deliberately under-specified at that level of abstraction [54].

7.3. Bounded Model-Checking

The applications described in the previous subsection are about *concrete* scenarios – whether actual, historical, or hypothetical – and may involve the enumeration of large sets of scenarios, e.g., when testing or running simulations. In other work we applied bounded model-checking to demonstrate the **verification** of safety and liveness properties [34] via a translation to nuXmv [25]. In these experiments we reason about *abstract* scenarios, using, for example, linear temporal logic to describe scenarios mixing concrete details and abstract placeholders. To apply bounded model-checking, eFLINT specifications must have **finite domains** such that in every runtime state, only a finite amount of transitions are possible. Bounded model-checking also requires the **closed world** assumption.

A goal of this experiment was to show that model-checking can be used in combination with the aforemen-

tioned applications, i.e., to verify certain properties before deploying a specification in a case management or data exchange system, without having to maintain multiple versions of the specification (static, **ex-ante enforcement**). To achieve this, we can rely on eFLINT’s features to extend a specification by closing open types and **reducing domains to finite domains**. This way, the same specification can be extended for model-checking and for deployment.

In future work we wish to demonstrate that we can use model checking for static regulatory compliance checking as described in Section 2.2 by taking advantage of the separation between physical and institutional actions in eFLINT. In addition, we aim to investigate the applicability of combinatorial, property-based testing for eFLINT [52].

7.4. Limitations and Future Work

The various experiments discussed in the previous subsections demonstrate the expressive capability of the language, including the suitability of the **datatypes** offered for knowledge representation. For specific applications, additional primitive types and operators may be needed, such as real or floating point numbers and operators for string manipulation. Future versions of the language may add additional types and operators, although potentially at the cost that not all implementations offer support. The semantics of eFLINT can be made more expressive by considering continuous elements, e.g., to represent clocks, and probabilistic reasoning.

Pragmatic design decisions have been made to ensure the language is sufficiently flexible to be used for various kinds of applications. However, a flexible solution is not always the most user-friendly solution. For example, in some applications a strict separation between specification (type declarations) and scenarios (statements) is useful, and the closed world assumption and finite domains are needed for model checking. In these applications, the necessary constraints need to be *assumed*. Conversely, a variant of the language dedicated to an application could *enforce* the constraints syntactically or using static analyses. For security reasons, it would also be better to explicitly hide/expose types using a module system (**service interface**).

At present, the language is not sufficiently friendly for use by legal experts without software expertise. Experience with computer science students does suggest that the language is relatively easy to learn for those with programming experience, especially declarative programming experience. Both language implementations are also lacking a mechanism to report how facts came to be true (or false). Ideally, such a report contains the sequence of rules and statements that were applied to render the fact true (or false). This feature is difficult to implement due to reasoning by default (e.g., in case of negation) and the subtle interplay between derivation rules and state transitions. The information from such a report is essential to achieve **explainability**. In Section 5 we discussed the use of an execution graph in the reference interpreter to maintain execution history

as an essential, yet less fine-grained, contributor to explainability. Note that in general, however, explainability requires the tailoring of information to specific receivers, tying the requirement together with the **domain-users** requirement. That is, legal and software experts appreciate a different perspective on the low-level system information.

An open question still being investigated is how the **interpretation** process can best be supported, especially to deal with the amount of existing laws and regulations and the pace with which they are produced. The current FLINT Rule Editor deploys a pipeline in which natural language processing (NLP) produces FLINT frames from a normative source. A legal expert then completes, refines and verifies the generated FLINT frames. A software expert completes and refines the eFLINT code automatically generated from the FLINT frames by using eFLINT’s extension mechanism. In a new project we will experiment with the use of generative AI within this pipeline.

Software expertise is needed, in particular, to refine the eFLINT code in order to formalise the connection between the eFLINT specification and the software environment in which the specification will integrate, e.g., using qualification rules, instrumentation, scripting and/or API adaptation. Software expertise is also needed to add computational details not easily derived from the natural language sources, such as quantification and constraints on the possible substitutions of variables. For consistency, however, it may be preferred for both experts to work on shared artefacts written in the same language.

As possible future work, we envisage a norm specification language which comprises an intermediate language and one or more legal surface-level languages. The experiences gained during the development of eFLINT, reported in this paper, make it possible to identify a minimal computational core that is complete with respect to the requirements. One motivation behind such a (re-)design is improving usability for legal and domain experts by offering a more tailored experience and potentially allowing both experts to work on the same specification. For example, a keyword such as **Syncs with** should be hidden as we should not require legal experts to think about transition system semantics. A rule-based syntax can be used to express the relation between **raise-hand** and **place-bid** without **Syncs with** as follows:

```
raise-hand(bidder) QualifiesAs
  place-bid(bidder, display.object, price)
  Where price = ...

... is equivalent to ...

Extend raise-hand Syncs with place-bid
(bidder=actor, object=display.object, price = ...)
```

We can go further by enabling such rules to be expressed in a controlled natural language such as a variant of Logical English, developed by Kowalski for, among other things, coding legal rules as logic programs [77]. Another example is RegelSprak, developed by the Dutch tax administration to formalise parts of the Dutch tax law [33].

To support a wide range of user-facing languages, additional **normative concepts** need to be mapped to computational concepts. For example, eFLINT equates a prohibition to act with the negation of a permission to act. Rather than forcing this interpretation, an intermediate language could simultaneously support explicit permissions and prohibitions. Permissions and prohibitions can then conflict, and a priority mechanism is needed to resolve such conflicts. For example, permissions and prohibitions may be assigned by actors with different levels of authority or may originate from different sources of norms that differ in precedence or specificity. FLINT and eFLINT currently lack an explicit mechanism for conflict resolution (**norm priorities**). The eFLINT semantics give an implicit priority to prohibitions over permissions, and to powers over prohibitions, due to how violations are generated and the effects of actions manifest.

A formal operational semantics and a type system would enable a more precise and detailed discussion on the language’s design, its properties and possible variants. At present, neither exists for eFLINT-v4. In [42], a formal semantics has been given to eFLINT-clingo.

8. Related work

In this section we describe work related to automating compliance and compare eFLINT to existing languages. Several software languages and logics exist to formalise and reason with norms from various sources and within different application areas. The language and logics differ primarily in their support for fundamental normative concepts, the types of legal documents targeted by the language, the means of execution, and the intended applications.

Normative concepts. Relative to other languages, eFLINT is most similar to languages based on the event calculus [76, 99, 98, 28] such as Symboleo [105] and InstAL [90]. Like eFLINT(-clingo), the Institutional Action Language (InstAL) is a DSL for specifying norms in terms of duties and powers translating to Answer Set Programming (ASP) for execution. The language has been primarily proposed for modelling institutions and reasoning within multi-agent systems, whereas eFLINT, in addition, has been developed to integrate in software systems and to reason about regulatory compliance of software.

Symboleo and eFLINT are both based on Hohfeld’s legal framework [60] that emphasises normative relations between actors centred around the normative concepts of power and duty. eFLINT is designed to specify norms from a wide range of legal documents, whereas Symboleo is designed specifically for contracts, embedding notions of contract state in the language. Instead of relying on built-in state predicates, an eFLINT user could define such predicates themselves. The original implementation of Symboleo was also based on logic programming (in Prolog, see [105]) and the language has since also been implemented

to admit model checking [93], access control [4] and to generate smart contracts [95].

Other formal languages for expressing norms are directly based on modal logics such as deontic logic [59] and defeasible logic [87, 55]. Deontic logics have been studied extensively for normative reasoning [64, 47]. Deontic logics admit modalities (operators) for expressing the permission, prohibition and obligation of propositions built from predicates. An appealing aspect of deontic logics is that applications of the operators can be nested, making it possible to capture and compare subtle differences in interpretations. Moreover, conflicts between permissions and prohibitions can be explicitly expressed. A downside is the absence of actions, events and temporal reasoning.

For temporal reasoning, an eFLINT specification defines a process model through its action- and event-type declarations. As such, it is possible to represent changes in facts and norms, and therefore to formalise the normative concept of power. A benefit of eFLINT's event-based approach is that checking the compliance of software systems is simplified because systems are inherently event-based.

The lack of temporal reasoning in deontic logics is addressed by Deontic Action Logics (DALs) shifting the perspective from deontic operators on predicates to deontic operators on actions [8]. Actions can be composed via parallel or free-choice composition operators. Permission and prohibition operators are used to determine under which conditions actions are permitted or prohibited.

LegalRuleML extends RuleML as a markup language for representing and exchanging legal rules based on deontic and defeasible logic. An interpretation of LegalRuleML rules in terms of a defeasible logic has been given in [79]. As part of future work, it would be meaningful to demonstrate to which extent eFLINT can be used as a back-end for executing LegalRuleML rules. In [106], parts of LegalRuleML have been translated to a custom intermediate language to enable reasoning with multiple automated theorem provers with different levels of expressivity.

Temporal logics such as Linear Temporal Logic (LTL) and Computation-Tree Logic (CTL) have been combined with concepts for normative reasoning. For example, Normative Temporal Logic (NTL) is a temporal logic that replaces the path quantifiers of Computation-Tree Logic (CTL) to express obligations and permissions [116]. Temporal Defeasible Logic (TDL) combines defeasibility and temporal logic [56]. The FIEVeL specification language is used to model institutional policies [113] based on Ordered Many-Sorted First-Order Temporal Logic (OMSFO TL) and uses SPIN [62] for model checking. In [10], the connection between coordination problems and norm enforcement is formalised using LTL. The language Revani [71] uses CTL for the specification of privacy norms in the context of sociotechnical systems.

Computational law. The languages Catala [83] and Regel-Spraak [33] are explicitly targetted for computational law, such as tax laws or social benefits laws. Computational law

is the subset of the law with statutes describing (potentially complex) computations. Both languages are explicitly designed to admit co-development of specifications by lawyers and programmers. The languages are used as part of legal expert systems [66], sharing our motivation of enabling automated governance (see Section 7.1).

Focussed on computational law, both languages do not directly embed normative concepts such as permission, obligation or power. A specification essentially describes a collection of functions defined through rules capturing the intricacies of the computations embedded in laws. The emphasis is on explicating implicit elements and identifying and resolving ambiguities in the law. In Catala, specific attention is given to the modular and literate development of a specification, allowing earlier definitions to be supplemented by later definitions through an exception mechanism, comparable to eFLINT's type extensions. The lessons learnt – such as on co-development and working with dates [85] – can inspire next versions of eFLINT.

Contracts. Contracts form a subset of the law that is characterised by a number of properties that are particularly amenable to automation. A contract encodes an agreement between two or more parties with an explicitly defined scope and lays out how the normative positions of the involved parties evolve over time, typically in response to events. As such, a contract can naturally be formalised by a state machine. This observation forms the basis underneath *smart contracts*.

First introduced by Szabo [108] for the exchange of digital assets, smart contracts are now most popular for their usage as scripts executing transactions recorded on a distributed ledger (the blockchain). Owing to their execution model, smart contracts can guarantee that the exchange of certain digital assets proceeds according to rules encoded in the smart contracts. Several blockchain platforms exist, often with their own language for developing smart contracts. But despite their name, smart contract languages are not designed for automating legal compliance and do not support normative concepts directly [5, 82]. Smart contract languages can form an interesting compilation target for normative specification languages, however. After all, using the ledger, blockchain applications can provide auditability and transparency [94, 104, 109]. As mentioned earlier, Symboleo contracts translate to smart contracts [95]. In [9], Azzopardi et al. describe an approach for monitoring deontic concepts using smart contracts.

Access and Usage Control. A significant body of work exists concerning the formalisation, analysis and enforcement of specific kinds of policies [68] such as access control policies [88, 67] and network policies [3]. The Margrave Policy Analyzer tool⁸ supports multiple formalisms to reason about such policies. For example, change-impact analysis

⁸<http://www.margrave-tool.org/>

makes the impact of changes to policies insightful [45] and would be useful to normative specification languages.

Many access control models exist [101, 89]. The most common are Attribute-Based Access Control (ABAC) [65], Role-Based Access Control (RBAC) [100], and Purpose-Based Access Control (PBAC) [23]. The knowledge representation of eFLINT is sufficiently expressive to capture policies based on roles, attributes and purposes.

XACML is a popular ABAC language and the XACML architecture [88] is widespread as a model for systems integrating ABAC or other forms of access control. The primary feature of the XACML architecture is its separation of concerns, making it possible for a system administrator to modify policies without other changes to the system. An eFLINT reasoner can be used as a policy decision point within the XACML architecture, as demonstrated in [16, 14].

Whereas access control can enforce permissions and prohibitions, usage control models take a step towards the enforcement of obligations. That is, policies can specify invariants that need to be maintained for the duration of a data processing action [92]. However, many usage conditions in data processing agreements, such as retention periods, cannot be enforced this way, as they survive beyond the processing activity. ODRL is popular as a standard for defining usage control policies controlling specific actions on multi-media assets [67, 96, 114]. As a representation language, ODRL does not have a fixed semantics⁹. Although ODRL embeds the concept ‘obligation’, it is rather a permission from the normative perspective [73].

9. Conclusion

This paper has reflected on the design and implementations of eFLINT based on requirements extracted from experiments in various application areas. The main novelty of the language is the orientation of the language towards the formalisation of a wide range of normative sources, including many classes of legal documents such as regulations, laws and contracts. A specification in the language defines data-types for knowledge representation, process modelling and establishes norms by applying the language’s formalisation of the normative concepts of permission, prohibition, obligation and power. Through specific language constructs, normative sources can be formalised at their own level of abstraction, reused, interconnected and integrated into software systems. The language’s flexibility allows it to be used across application areas such as static and dynamic regulatory compliance checking, auditing, and simulation. The flexibility is achieved through certain pragmatic design decisions and by embedding software engineering principles such as modularity and extensibility. The main future developments are to increase the

usability of the language, in particular for legal experts, by developing new ways of interacting with the language, including alternative surface-level languages, static analyses, back-ends, and editors.

Acknowledgments

Special thanks go to all the students, researchers, and domain experts that have provided feedback over the years. Additional thanks to the anonymous reviewers for their useful suggestions to improving this paper.

This work has been executed as part of the SSPDDP project supported by NWO (628.009.014), the AMdEX Fieldlab project supported by Kansen Voor West EFRO (KVV00309) and the province of Noord-Holland, and the DMI ecosystem supported by the National Growth Fund.

References

- [1] Aagesen, G., Krogstie, J.: BPMN 2.0 for Modeling Business Processes, pp. 219–250. Springer Berlin Heidelberg, Berlin, Heidelberg (2015). https://doi.org/10.1007/978-3-642-45100-3_10
- [2] van der Aalst, W.M.P.: Conformance Checking, pp. 191–213. Springer Berlin Heidelberg, Berlin, Heidelberg (2011). https://doi.org/10.1007/978-3-642-19345-3_7
- [3] Al-Shaer, E.S., Hamed, H.H.: Modeling and management of firewall policies. *IEEE Transactions on Network and Service Management* **1**(1), 2–10 (2004). <https://doi.org/10.1109/TNSM.2004.4623689>
- [4] Alfuhaid, S., Anda, A.A., Amyot, D., Roveri, M., Mylopoulos, J.: Symboleoac: An access control model for legal contracts. In: Paja, E., Zdravkovic, J., Kavakli, E., Stirna, J. (eds.) *The Practice of Enterprise Modeling*. pp. 227–243. Springer Nature Switzerland, Cham (2025). https://doi.org/10.1007/978-3-031-77908-4_14
- [5] Allen, J.: Wrapped and stacked: ‘smart contracts’ and the interaction of natural and formal language. *European Review of Contract Law* **14**(4), 307–343 (2018). <https://doi.org/doi:10.1515/ercl-2018-1023>
- [6] Alsayed Kassem, J., Allaart, C., Amiri, S., Kebede, M., Müller, T., Turner, R., Belloum, A., van Binsbergen, L.T., Grunwald, P., van Halteren, A., Grosso, P., de Laat, C., Klous, S.: Building a Digital Health Twin for Personalized Intervention: The EPI Project. In: Haverkort, B.R., de Jongste, A., van Kuilenburg, P., Vromans, R.D. (eds.) *Commit2Data. Open Access Series in Informatics (OASIs)*, vol. 124, pp. 2:1–2:18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2024). <https://doi.org/10.4230/OASIs.Commit2Data.2>

⁹A working group is specifying semantics for ODRL here: <https://w3c.github.io/odrl/formal-semantics/>.

- [7] Alsayed Kassem, J., Müller, T., Esterhuyse, C.A., Kebede, M.G., Osseyran, A., Grosso, P.: The EPI framework: A data privacy by design framework to support healthcare use cases. *Future Generation Computer Systems* **165**, 107550 (2025). <https://doi.org/https://doi.org/10.1016/j.future.2024.107550>, <https://www.sciencedirect.com/science/article/pii/S0167739X24005144>
- [8] Areces, C., Cassano, V., Castro, P.F., Fervari, R.: Deontic action logics: A modular algebraic perspective. *J of Log Lang and Inf* (2025). <https://doi.org/10.1007/s10849-025-09447-7>
- [9] Azzopardi, S., Pace, G.J., Schapachnik, F.: On observing contracts: Deontic contracts meet smart contracts. In: Palmirani, M. (ed.) *Legal Knowledge and Information Systems - JURIX 2018: The Thirty-first Annual Conference*, Groningen, The Netherlands, 12-14 December 2018. pp. 21–30. *Frontiers in Artificial Intelligence and Applications*, IOS Press (2018). <https://doi.org/10.3233/978-1-61499-935-5-21>, <https://doi.org/10.3233/978-1-61499-935-5-21>
- [10] Aștefănoaei, L., de Boer, F., Dastani, M., Meyer, J.J.: On the Semantics and Verification of Normative Multi-Agent Systems **15**(13), 2629–2652. <https://doi.org/10.3217/jucs-015-13-2629>
- [11] Bahig, H.M., Hazber, M.A.G., Al-Utaibi, K.A., Nassr, D.I., Bahig, H.M.: Efficient sequential and parallel prime sieve algorithms. *Symmetry* **14**(12), 2527 (2022). <https://doi.org/10.3390/SYM14122527>, <https://doi.org/10.3390/sym14122527>
- [12] Bartocci, E., Falcone, Y., Francalanza, A., Reger, G.: Introduction to Runtime Verification. In: Bartocci, E., Falcone, Y. (eds.) *Lectures on Runtime Verification*, vol. 10457, pp. 1–33. Springer International Publishing, Cham (2018). https://doi.org/10.1007/978-3-319-75632-5_1, series Title: *Lecture Notes in Computer Science*
- [13] van Binsbergen, L.T.: Haskell prototype implementation of the eflint language. <https://gitlab.com/eflint/haskell-implementation> (2020), [Online, accessed 01 May 2026]
- [14] van Binsbergen, L.T., Kebede, M.G., Baugh, J., van Engers, T., van Vuurden, D.G.: Dynamic generation of access control policies from social policies. *Procedia Computer Science* **198**, 140–147 (January 2022). <https://doi.org/10.1016/j.procs.2021.12.221>
- [15] van Binsbergen, L.T., Liu, L., van Doesburg, R., van Engers, T.: eFLINT: A Domain-Specific Language for Executable Norm Specifications. In: *Proceedings of the 19th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences*. pp. 124–136. GPCE 2020, ACM (2020). <https://doi.org/10.1145/3425898.3426958>
- [16] van Binsbergen, L.T., Oost-Rosengren, M., Schreijer, H., Dijkstra, F., van Dijk, T.: AMdEX Reference Architecture – version 1.0.0. Zenodo (2 2024). <https://doi.org/10.5281/zenodo.10565915>
- [17] van Binsbergen, L.T., Scott, E., Johnstone, A.: GLL parsing with flexible combinators. In: *Proceedings of the 11th ACM SIGPLAN International Conference on Software Language Engineering*. SLE 2018, ACM (2018). <https://doi.org/10.1145/3276604.3276618>
- [18] van Binsbergen, L.T., Scott, E., Johnstone, A.: Purely Functional GLL Parsing. *Journal of Computer Languages* (2020). <https://doi.org/10.1016/j.cola.2020.100945>
- [19] van Binsbergen, L.T., Verano Merino, M., Jeanjean, P., van der Storm, T., Combemale, B., Barais, O.: A Principled Approach to REPL Interpreters, pp. 84–100. ACM (2020). <https://doi.org/10.1145/3426428.3426917>
- [20] Bonfanti, S., Riccobene, E., Scandurra, P.: A component framework for the runtime enforcement of safety properties. *Journal of Systems and Software* **198**, 111605 (2023). <https://doi.org/10.1016/j.jss.2022.111605>
- [21] Breebaart, M.: Juriconnect standaard BWB versie 1.3.1 (Oct 2014)
- [22] Breteler, J., van Gessel, T., Biagioni, G., van Doesburg, R.: The FLINT ontology: An actor-based model of legal relations. In: Acosta, M., Peroni, S., Vahdati, S., Gentile, A.L., Pellegrini, T., Kalo, J. (eds.) *Knowledge Graphs: Semantics, Machine Learning, and Languages - Proceedings of the 19th International Conference on Semantic Systems*, 20-22 September 2023, Leipzig, Germany. pp. 227–234. *Studies on the Semantic Web*, IOS Press (2023). <https://doi.org/10.3233/SSW230016>
- [23] Byun, J.W., Bertino, E., Li, N.: Purpose based access control of complex data for privacy protection. In: *Proceedings of the tenth ACM symposium on Access control models and technologies*. pp. 102–110 (2005). <https://doi.org/10.1145/1063979.1063998>, <https://doi.org/10.1145/1063979.1063998>
- [24] Calimeri, F., Gebser, M., Maratea, M., Ricca, F.: Design and results of the fifth answer set programming competition. *Artif. Intell.* **231**, 151–181 (2016)
- [25] Cavada, R., Cimatti, A., Dorigatti, M., Griggio, A., Mariotti, A., Micheli, A., Mover, S., Roveri, M., Tonetta, S.: The nuXmv Symbolic Model Checker.

- In: Biere, A., Bloem, R. (eds.) *Computer Aided Verification*. pp. 334–342. Lecture Notes in Computer Science, Springer International Publishing. https://doi.org/10.1007/978-3-319-08867-9_22
- [26] Ceci, M., Sannier, N., Abualhaija, S., Shin, D., Bianculli, D., Halling, M.: Toward Automated Compliance Checking of Fund Activities Using Runtime Verification Techniques. In: *Proceedings of the 1st IEEE/ACM Workshop on Software Engineering Challenges in Financial Firms*. pp. 19–20. ACM, Lisbon Portugal (Apr 2024). <https://doi.org/10.1145/3643665.3648045>
- [27] Ceri, S., Gottlob, G., Tanca, L.: What you always wanted to know about datalog (and never dared to ask). *IEEE Trans. Knowl. Data Eng.* **1**(1), 146–166 (1989). <https://doi.org/10.1109/69.43410>
- [28] Charalambides, M., Flegkas, P., Pavlou, G., Bandara, A.K., Lupu, E., Russo, A., Dulay, N., Sloman, M., Rubio-Loyola, J.: Policy conflict analysis for quality of service management. In: *6th IEEE International Workshop on Policies for Distributed Systems and Networks (POLICY 2005)*, 6–8 June 2005, Stockholm, Sweden. pp. 99–108. IEEE (2005). <https://doi.org/10.1109/POLICY.2005.23>
- [29] Clark, K.L.: Negation as failure. In: Gallaire, H., Minker, J. (eds.) *Logic and Data Bases, Symposium on Logic and Data Bases, Centre d’études et de recherches de Toulouse, France, 1977*. pp. 293–322. *Advances in Data Base Theory*, Plenum Press, New York (1977). https://doi.org/10.1007/978-1-4684-3384-5_11
- [30] Clarke, E.M.: *Model Checking*. MIT Press, 2 edn.
- [31] Clarke, E.M., Henzinger, T.A., Veith, H., Bloem, R. (eds.): *Handbook of Model Checking*. Springer (2018). <https://doi.org/10.1007/978-3-319-10575-8>
- [32] Colmerauer, A., Roussel, P.: The Birth of Prolog, p. 331–367. Association for Computing Machinery, New York, NY, USA (1996). <https://doi.org/10.1145/234286.1057820>
- [33] Corsius, M., Hoppenbrouwers, S., Lokin, M., Baars, E., Sangers-Van Cappellen, G., Wilmont, I.: *Regel-Spraak: a CNL for Executable Tax Rules Specification*. In: Kuhn, T., Spreeuwenberg, S., Hoppenbrouwers, S., Fuchs, N.E. (eds.) *Proceedings of the Seventh International Workshop on Controlled Natural Language (CNL 2020/21)*. Special Interest Group on Controlled Natural Language, Amsterdam, Netherlands (Sep 2021), <https://aclanthology.org/2021.cnl-1.6/>
- [34] De Geus, F.W.: *Model Checking Normative Systems*. Master’s thesis, University of Amsterdam (2022)
- [35] Dell’Anna, D., Dastani, M., Dalpiaz, F.: Runtime revision of norms and sanctions based on agent preferences. *Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems, AAMAS* **3**, 1609–1617 (2019). <https://doi.org/10.65109/GQPO3105>
- [36] van Doesburg, R., van Engers, T.: The false, the former, and the parish priest. In: *Proceedings of the Seventeenth International Conference on Artificial Intelligence and Law*. pp. 194–198. ICAIL 2019, ACM (2019). <https://doi.org/10.1145/3322640.3326718>
- [37] van Doesburg, R., van der Storm, T., van Engers, T.: CALCULEMUS: Towards a formal language for the interpretation of normative systems. In: *AI4J Workshop at ECAI 2016*. pp. 73–77. AI4J 2016 (2016)
- [38] Erdem, E., Gelfond, M., Leone, N.: Applications of answer set programming. *AI Magazine* **37**(3), 53–68 (2016). <https://doi.org/10.1609/aimag.v37i3.2678>
- [39] Esterhuyse, C.A.: *Specification-centric Multi Agent Systems*. Ph.D. thesis, University of Amsterdam (2025)
- [40] Esterhuyse, C.A., van Binsbergen, L.T.: Cooperative Specification via Composition Control. In: *Proceedings of the 17th ACM SIGPLAN International Conference on Software Language Engineering*. p. 2–15. SLE ’24, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3687997.3695635>
- [41] Esterhuyse, C.A., Müller, T., van Binsbergen, L.T.: JustAct: Actions Universally Justified by Partial Dynamic Policies. In: Castiglioni, V., Francalanza, A. (eds.) *Formal Techniques for Distributed Objects, Components, and Systems*. pp. 60–81. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-62645-6_4
- [42] Esterhuyse, C.A., Müller, T., van Binsbergen, L.T.: A Stable Model Semantics for eFLINT Norm Specifications and Model Checking Scenarios. In: *Proceedings of the 24th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences*. p. 80–93. GPCE ’25, Association for Computing Machinery, New York, NY, USA (2025). <https://doi.org/10.1145/3742876.3742882>
- [43] Esterhuyse, C.A., Müller, T., Van Binsbergen, L.T., Belloum, A.S.Z.: Exploring the enforcement of private, dynamic policies on medical workflow execution. In: *2022 IEEE 18th International Conference on e-Science (e-Science)*. pp. 481–486 (2022). <https://doi.org/10.1109/eScience55777.2022.00086>
- [44] Ethereum: Solidity documentation online. <https://solidity.readthedocs.io> (2016), [Online, accessed 23 October 2025]

- [45] Físlér, K., Krishnamurthi, S., Meyerovich, L., Tschantz, M.: Verification and Change-Impact Analysis of Access-Control Policies. In: Proceedings of the 27th International Conference on Software Engineering. pp. 196–205. ICSE 2005, Association for Computing Machinery, New York, NY, USA (2005). <https://doi.org/10.1145/1062455.1062502>
- [46] Frölich, D., van Binsbergen, L.T.: A Generic Back-End for Exploratory Programming. In: The 22nd International Symposium on Trends in Functional Programming (TFP 2021). LNCS, vol. 12834. Springer (2021). https://doi.org/10.1007/978-3-030-83978-9_2
- [47] Gabbay, D., Horty, J., Parent, X.: Handbook of Deontic Logic and Normative Systems. College Publications (2013)
- [48] Gebser, M., Kaminski, R., Kaufmann, B., Ostrowski, M., Schaub, T., Wanko, P.: Theory solving made easy with clingo 5. In: Carro, M., King, A., Saeedloei, N., Vos, M.D. (eds.) Technical Communications of the 32nd International Conference on Logic Programming, ICLP 2016 TCs, October 16–21, 2016, New York City, USA. OASICS, vol. 52, pp. 2:1–2:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2016). <https://doi.org/10.4230/OASICS.ICLP.2016.2>
- [49] Gebser, M., Kaminski, R., Kaufmann, B., Schaub, T.: Multi-shot ASP solving with clingo. Theory Pract. Log. Program. **19**(1), 27–82 (2019). <https://doi.org/10.1017/S1471068418000054>
- [50] Gebser, M., Kaufmann, B., Kaminski, R., Ostrowski, M., Schaub, T., Schneider, M.: Potassco: The potsdam answer set solving collection. AI Communications **24**(2), 107–124 (2011). <https://doi.org/10.3233/AIC-2011-0491>
- [51] Gelfond, M., Lifschitz, V.: The stable model semantics for logic programming. In: Kowalski, R.A., Bowen, K.A. (eds.) Logic Programming, Proceedings of the Fifth International Conference and Symposium, Seattle, Washington, USA, August 15–19, 1988 (2 Volumes). pp. 1070–1080. MIT Press (1988)
- [52] Goldstein, H., Hughes, J., Lampropoulos, L., Pierce, B.C.: Do Judge a Test by its Cover. In: Yoshida, N. (ed.) Programming Languages and Systems. pp. 264–291. Springer International Publishing (2021). https://doi.org/10.1007/978-3-030-72019-3_10
- [53] Gommans, L., Xu, L., Demchenko, Y., Wan, A., Cristea, M., Meijer, R., de Laat, C.: Multi-domain lighthouse authorization, using tokens. Future Generation Computer Systems **25**(2), 153–160 (2009). <https://doi.org/10.1016/j.future.2008.07.013>
- [54] Governatori, G., Idelberger, F., Milosevic, Z., Riveret, R., Sartor, G., Xu, X.: On legal contracts, imperative and declarative smart contracts, and blockchain systems. Artificial Intelligence and Law **26**(4), 377–409 (2018). <https://doi.org/10.1007/s10506-018-9223-3>
- [55] Governatori, G., Maher, M., Antoniou, G., Billington, D.: Argumentation Semantics for Defeasible Logic. Journal of Logic and Computation **14**(5), 675–702 (2004). <https://doi.org/10.1093/logcom/14.5.675>
- [56] Governatori, G., Rotolo, A.: Changing legal systems: Legal abrogations and annulments in Defeasible Logic **18**(1), 157–194. <https://doi.org/10.1093/jigpal/jzp075>
- [57] Groefsema, H., Beest, N.R.T.P.v., Governatori, G.: On the use of the conformance and compliance keywords during verification of business processes. In: Di Ciccio, C., Dijkman, R., del Río Ortega, A., Rinderle-Ma, S. (eds.) Business Process Management Forum. pp. 21–37. Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-031-16171-1_2
- [58] Hashmi, M., Governatori, G., Lam, H., Wynn, M.T.: Are we done with business process compliance: state of the art and challenges ahead. Knowl. Inf. Syst. **57**(1), 79–133 (2018). <https://doi.org/10.1007/s10115-017-1142-1>
- [59] Herrestad, H.: Norms and formalization. In: Proceedings of the 3th International Conference on Artificial Intelligence and Law. pp. 175–184. ICAIL 1993, ACM (1993). <https://doi.org/10.1145/112646.112667>
- [60] Hohfeld, W.N.: Fundamental legal conceptions as applied in judicial reasoning. The Yale Law Journal **26**(8), 710–770 (1917). <https://doi.org/10.2307/786270>
- [61] Hohfeld, W.: Some fundamental legal conceptions as applied in judicial reasoning. Yale Law Journal **23**(1), 59–64 (1913)
- [62] Holzmann, G.J.: The SPIN Model Checker - primer and reference manual. Addison-Wesley (2004)
- [63] Horrocks, I., Patel-Schneider, P.F., McGuinness, D.L., Welty, C.A.: OWL: a Description-Logic-Based Ontology Language for the Semantic Web, p. 458–486. Cambridge University Press (2007)
- [64] Horty, J.: Deontic modals: Why abandon the classical semantics? Pacific Philosophical Quarterly **95**(4), 424–460 (2014)
- [65] Hu, V.C., Kuhn, D.R., Ferraiolo, D.F.: Attribute-based access control. Computer **48**(2), 85–88 (2015). <https://doi.org/10.1109/MC.2015.33>

- [66] Huttner, L., Merigoux, D.: Catala: Moving towards the future of legal expert systems. *Artificial Intelligence and Law* (2022). <https://doi.org/10.1007/s10506-022-09328-5>
- [67] Iannella, R., Villata, S.: ODRL information model 2.2. W3C Recommendation (2018)
- [68] Jabal, A., Davari, M., Bertino, E., Makaya, C., Calo, S., Verma, D., Russo, A., Williams, C.: Methods and tools for policy analysis. *ACM Computing Surveys* **51**(6) (2019). <https://doi.org/10.1145/3295749>
- [69] Jackson, D.: *Software Abstractions: Logic, Language, and Analysis*. The MIT Press (2006)
- [70] Jones, A., Sergot, M.: A Formal Characterisation of Institutionalised Power. *Logic Journal of the IGPL* **4**(3), 427–443 (06 1996). <https://doi.org/10.1093/jigpal/4.3.427>
- [71] Kafalý, O., Ajmeri, N., Singh, M.P.: Revani: Revising and Verifying Normative Specifications for Privacy **31**(5), 8–15. <https://doi.org/10.1109/MIS.2016.89>
- [72] Karels, J.: Design of a Framework for Storing and Reasoning About Normative Sources with Formalised Interpretations. Master’s thesis, University of Amsterdam (2023)
- [73] Kebede, M.G., Sileno, G., Van Engers, T.: A Critical Reflection on ODRL. In: Rodríguez-Doncel, V., Palmirani, M., Araszkievicz, M., Casanovas, P., Pagallo, U., Sartor, G. (eds.) *AI Approaches to the Complexity of Legal Systems XI-XII*. pp. 48–61. Springer International Publishing, Cham (2021). https://doi.org/10.1007/978-3-030-89811-3_4
- [74] Kephart, J., Chess, D.: The vision of automatic computing. *Computer* **36**(1), 41–50 (2003). <https://doi.org/10.1109/MC.2003.1160055>
- [75] Klyne, G., Carroll, J.J.: Resource Description Framework (RDF): Concepts and Abstract Syntax. W3C Recommendation (2004), <http://www.w3.org/TR/2004/REC-rdf-concepts-20040210/>
- [76] Kowalski, R., Sergot, M.: A logic-based calculus of events. *New Generation Computing* **4**(1), 67–95 (1986). <https://doi.org/10.1007/BF03037383>, <https://doi.org/10.1007/BF03037383>
- [77] Kowalski, R., Dávila, J., Sartor, G., Calejo, M.: Logical English for Law and Education, pp. 287–299. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-35254-6_24
- [78] Krötzsch, M.: *OWL 2 Profiles: An Introduction to Lightweight Ontology Languages*, pp. 112–183. Springer Berlin Heidelberg, Berlin, Heidelberg (2012). https://doi.org/10.1007/978-3-642-33158-9_4
- [79] Lam, H.P., Hashmi, M., Scofield, B.: Enabling Reasoning with LegalRuleML. In: Alferes, J.J., Bertossi, L., Governatori, G., Fodor, P., Roman, D. (eds.) *Rule Technologies. Research, Tools, and Applications*. pp. 241–257. Springer International Publishing, Cham (2016). https://doi.org/10.1007/978-3-319-42019-6_16
- [80] Larman, C.: *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (3rd Edition). Prentice Hall PTR, USA (2004)
- [81] Liu, L.C., Parizi, M.M., van Binsbergen, L.T., van Engers, T.: Regulatory services to automate compliance with ex-post enforcement. In: *Proceedings of AI Approaches to the Complexity of Legal Systems (AICOL) 2023*. Springer (2024)
- [82] Liu, L., Sileno, G., van Engers, T.M.: Digital enforceable contracts (DEC): making smart contracts smarter **334**, 235–238 (2020). <https://doi.org/10.3233/FAIA200872>
- [83] Merigoux, D., Chataing, N., Protzenko, J.: Catala: a programming language for the law. *Proc. ACM Program. Lang.* **5**(ICFP) (Aug 2021). <https://doi.org/10.1145/3473582>
- [84] Mohun, J., Roberts, A.: Cracking the code: Rulemaking for humans and machines. Tech. Rep. 42, Organisation for Economic Co-operation and Development (OECD) (2020). <https://doi.org/10.1787/3afe6ba5-en>
- [85] Monat, R., Fromherz, A., Merigoux, D.: Formalizing date arithmetic and statically detecting ambiguities for the law. In: Weirich, S. (ed.) *Programming Languages and Systems*. pp. 421–450. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-57267-8_16
- [86] Müller, T., Esterhuyse, C.A., van Binsbergen, L.T.: Source code and experimental data for a translation from efflint normative specifications to clingo answer-set programs (2025). <https://doi.org/10.5281/zenodo.15188960>
- [87] Nute, D.: Defeasible logic. In: Bartenstein, O., Geske, U., Hannebauer, M., Yoshie, O. (eds.) *Web Knowledge Management and Decision Support*. pp. 151–169. Springer Berlin Heidelberg, Berlin, Heidelberg (2003). <https://doi.org/10.1007/3-540-36524-9>
- [88] OASIS eXtensible Access Control Markup Language (XACML) Technical Committee: eXtensible Access Control Markup Language (XACML) Version 3.0 Plus Errata 01 (July 2017)
- [89] Osborn, S.L.: Mandatory access control and role-based access control revisited. In: Youman, C.E.,

- Coyne, E.J., Jaeger, T. (eds.) *Proceedings of the Second Workshop on Role-Based Access Control, RBAC 1997*, November 6-7, 1997. pp. 31–40. ACM (1997). <https://doi.org/10.1145/266741.266751>
- [90] Padget, J., Elakehal, E., Li, T., De Vos, M.: *InstAL: An Institutional Action Language, Law, Governance and Technology Series*, vol. 30, p. 101. Springer Verlag (2016). https://doi.org/10.1007/978-3-319-33570-4_6
- [91] Parizi, M.M., van Binsbergen, L.T., Sileno, G., van Engers, T.: A Modular Architecture for Integrating Normative Advisors in MAS. In: Baumeister, D., Rothe, J. (eds.) *Multi-Agent Systems*. pp. 312–329. Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-031-20614-6_18
- [92] Park, J., Sandhu, R.: Towards usage control models: beyond traditional access control. In: *Proceedings of the Seventh ACM Symposium on Access Control Models and Technologies*. p. 57–64. SACMAT ’02, Association for Computing Machinery (2002). <https://doi.org/10.1145/507711.507722>
- [93] Parvizimosaed, A., Roveri, M., Rasti, A., Anda, A.A., Alfuhaid, S., Amyot, D., Logrippo, L., Mylopoulos, J.: SymboleoPC: checking properties of legal contracts. *Softw. Syst. Model.* **24**(4), 1093–1126 (2025). <https://doi.org/10.1007/S10270-024-01180-2>
- [94] Prinz, W., Rose, T., Urbach, N.: *Blockchain Technology and International Data Spaces*, pp. 165–180. Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-030-93975-5_10
- [95] Rasti, A., Anda, A.A., Alfuhaid, S., Parvizimosaed, A., Amyot, D., Roveri, M., Logrippo, L., Mylopoulos, J.: Automated generation of smart contract code from legal contract specifications with Symboleo2SC. *Softw. Syst. Model.* **24**(4), 1127–1156 (2025). <https://doi.org/10.1007/S10270-024-01187-9>
- [96] Rodríguez-Doncel, V., Villata, S., Gómez-Pérez, A.: A dataset of RDF licenses. In: Hoekstra, R. (ed.) *Legal Knowledge and Information Systems - JURIX 2014: The Twenty-Seventh Annual Conference*, Jagiellonian University, Krakow, Poland, 10-12 December 2014. *Frontiers in Artificial Intelligence and Applications*, vol. 271, pp. 187–188. IOS Press (2014). <https://doi.org/10.3233/978-1-61499-468-8-187>
- [97] Ross, K.A.: Modular stratification and magic sets for DATALOG programs with negation. In: Rosenkrantz, D.J., Sagiv, Y. (eds.) *Proceedings of the Ninth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems*, April 2-4, 1990, Nashville, Tennessee, USA. pp. 161–171. ACM Press (1990). <https://doi.org/10.1145/298514.298558>
- [98] Russo, A., Miller, R., Nuseibeh, B., Kramer, J.: An abductive approach for analysing event-based requirements specifications. In: Stuckey, P. (ed.) *Logic Programming, 18th International Conference, ICLP 2002, Copenhagen, Denmark, July 29 - August 1, 2002, Proceedings. LNCS*, vol. 2401, pp. 22–37. Springer (2002). https://doi.org/10.1007/3-540-45619-8_3
- [99] Sadri, F., Kowalski, R.: Variants of the event calculus. In: Sterling, L. (ed.) *Logic Programming, Proceedings of the Twelfth International Conference on Logic Programming, Tokyo, Japan, June 13-16, 1995*. pp. 67–81. MIT Press (1995)
- [100] Sandhu, R.S., Coyne, E.J., Feinstein, H.L., Youman, C.E.: Role-based access control models. *Computer* **29**(2), 38–47 (1996). <https://doi.org/10.1109/2.485845>
- [101] Sandhu, R.S., Munawar, Q.: How to do discretionary access control using roles. In: Youman, C.E., Jaeger, T. (eds.) *Proceedings of the Third ACM Workshop on Role-Based Access Control, RBAC*. pp. 47–54. ACM (1998). <https://doi.org/10.1145/286884.286893>
- [102] Schouten, M., Esterhuyse, C.A.: DMI Afsprakenstelsel (version of 23 April 2026) formalisation as an eFLINT specification with a set of test eFLINT scenarios. Zenodo (4 2026). <https://doi.org/10.5281/zenodo.19761312>
- [103] Searle, J.: *The construction of social reality*. Penguin Books (1996)
- [104] Shakeri, S., Maccatrozzo, V., Veen, L., Bakhshi, R., Gommans, L., de Laat, C., Grosso, P.: Modeling and matching digital data marketplace policies. In: *2019 15th International Conference on eScience (eScience)*. pp. 570–577 (2019). <https://doi.org/10.1109/eScience.2019.00078>
- [105] Sharifi, S., Parvizimosaed, A., Amyot, D., Logrippo, L., Mylopoulos, J.: Symboleo: Towards a Specification Language for Legal Contracts. In: Breaux, T.D., Zisman, A., Fricker, S., Glinz, M. (eds.) *28th IEEE International Requirements Engineering Conference, RE 2020, August 31 - September 4, 2020*. pp. 364–369. IEEE (2020). <https://doi.org/10.1109/RE48521.2020.00049>
- [106] Steen, A., Fuenmayor, D.: Bridging Between LegalRuleML and TPTP for Automated Normative Reasoning. In: Governatori, G., Turhan, A.Y. (eds.) *Rules and Reasoning*. pp. 244–260. Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-031-21541-4_16
- [107] Steketee, M., Verheijen, N., van Binsbergen, L.T.: Managing Administrative Law Cases using an Adaptable Model-driven Norm-enforcing Tool. *Proceed-*

ings of AI4Access2Justice at JURIX'24 (2024).
<https://doi.org/10.48550/arXiv.2510.24822>

- [108] Szabo, N.: Formalizing and securing relationships on public networks. *First Monday* **2**(9) (1997).
<https://doi.org/10.5210/fm.v2i9.548>
- [109] Tuler De Oliveira, M., Reis, L.H.A., Verginadis, Y., Mattos, D.M.F., Olabarriaga, S.D.: SmartAccess: Attribute-Based Access Control System for Medical Records Based on Smart Contracts. *IEEE Access* **10**, 117836–117854 (2022).
<https://doi.org/10.1109/ACCESS.2022.3217201>
- [110] Valkering, O., Cushing, R., Belloum, A.: Brane: A framework for programmable orchestration of multi-site applications. In: 2021 IEEE 17th International Conference on eScience (eScience). pp. 277–282 (2021).
<https://doi.org/10.1109/eScience51609.2021.00056>
- [111] van der Aalst, W.: Process mining: discovery, conformance and enhancement of business processes. Springer, Germany (2011).
<https://doi.org/10.1007/978-3-642-19345-3>
- [112] Van Gelder, A., Ross, K.A., Schlipf, J.S.: The well-founded semantics for general logic programs. *Journal of the ACM* **38**(3), 619–649 (Jul 1991).
<https://doi.org/10.1145/116825.116838>
- [113] Viganò, F., Colombetti, M.: Symbolic model checking of institutions. pp. 35–44. ACM Press.
<https://doi.org/10.1145/1282100.1282109>
- [114] Vos, M.D., Kirrane, S., Padget, J.A., Satoh, K.: ODRL Policy Modelling and Compliance Checking. In: Fodor, P., Montali, M., Calvanese, D., Roman, D. (eds.) *Rules and Reasoning - Third International Joint Conference, RuleML+RR 2019, Bolzano, Italy, September 16-19, 2019, Proceedings. Lecture Notes in Computer Science*, vol. 11784, pp. 36–51. Springer (2019). https://doi.org/10.1007/978-3-030-31095-0_3
- [115] Wood, D.: Ethereum: a secure decentralised generalised transaction ledger (2014)
- [116] Ågotnes, T., van der Hoek, W., Rodríguez-Aguilar, J.A., Sierra, C., Wooldridge, M.: A Temporal Logic of Normative Systems. In: Makinson, D., Malinowski, J., Wansing, H. (eds.) *Towards Mathematical Philosophy: Papers from the Studia Logica Conference Trends in Logic IV*, pp. 69–106. Springer Netherlands.
https://doi.org/10.1007/978-1-4020-9084-4_5